Hydrofrac: what ships, what we learned, which doors are closed

An inventory and retrospective of lineage-driven fault injection for Hydro, written against commit `2f156b6335` (bookmark `sandbox-6479bd3c-b09a-4092-b96d-1375d67833f6`, 2026-10-20).

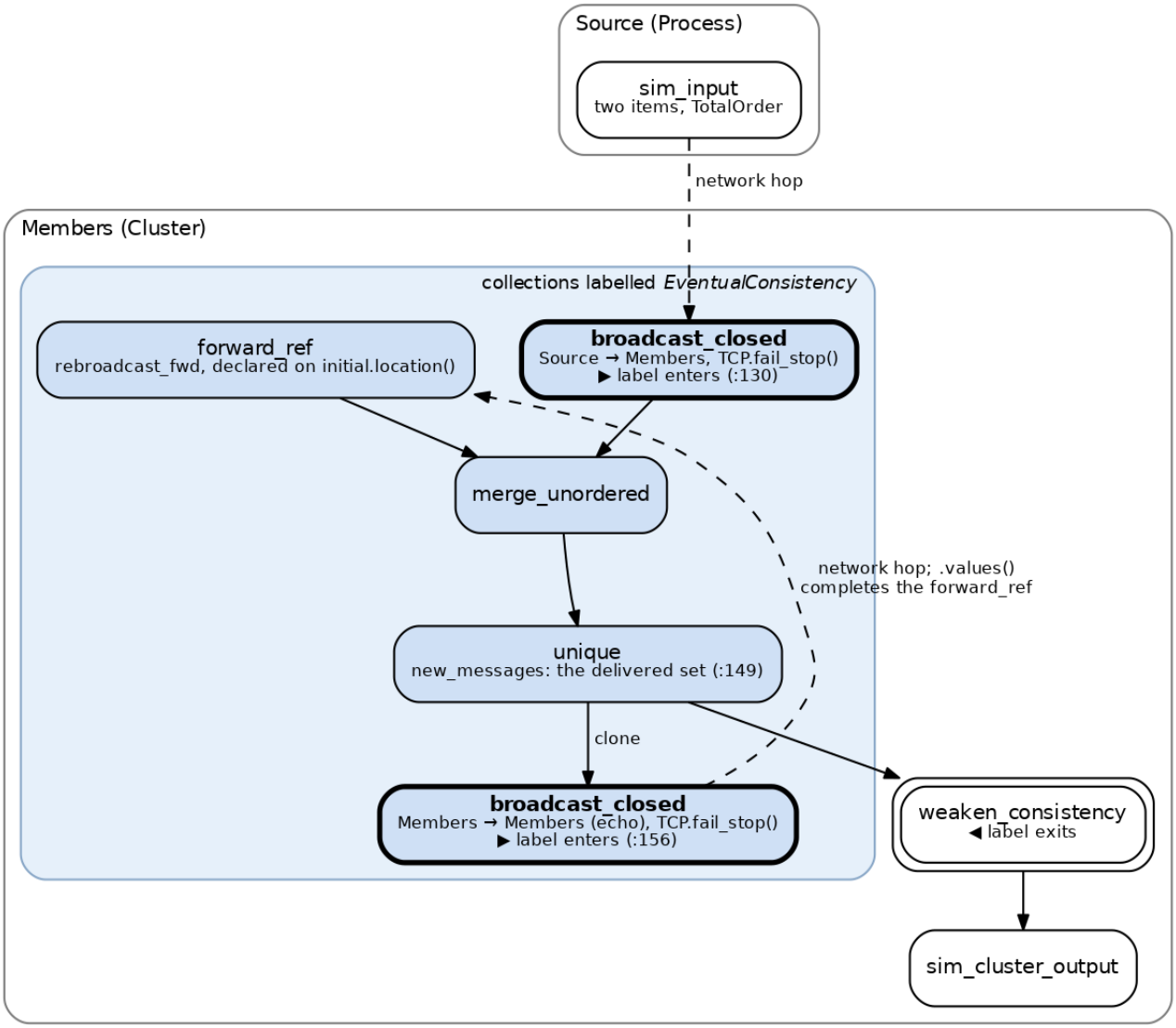
Hydro's type system certifies consistent computation: a collection labelled `EventualConsistency` is one whose contents at every cluster member are a deterministic function of the set of inputs that member received. (Sensitivity to delivery order and to retries is tracked by the type system's separate ordering and retry parameters and is not the label's concern.) The label must therefore assume that the input sets are the same: it says nothing useful about two members whose inputs differ, so the guarantee it offers is conditional on an agreement among inputs that the type system never checks. A broadcaster that dies between the second and third send of one fan-out leaves two members with different inputs, and each member's contents are then perfectly consistent with its own inputs and different from its neighbour's — the label holds and the members disagree. Hydrofrac discharges that assumed premise. It runs the program in the Hydro simulator, injects the crashes and omissions the program's own transport declarations admit, and searches, lineage-first in the manner of Molly (Alvaro, Rosen, Hellerstein, SIGMOD 2015), for an execution in which a fact held by one correct member at the end of the run is absent at another. What converged, after six rounds, is a two-level tool (`Level::Execution` and `Level::Precise`), with fault points at the sender on every network send and a fault universe grown lazily from the derivations the search has seen. The last round's experiments — a four-level lineage ablation, disjunctive provenance, aggregate-local witnesses — mostly closed doors rather than opening them, and that is why I believe the design has converged rather than merely stopped. Part I is the automatic path and stands alone: what `all_labels` checks, how to run it, what a verdict is worth. Part II is the hand-written path and what complicates it. Part III is what the search's experiments taught. Part IV is the doors.

Part I — Validating `EventualConsistency`

1. What it checks and how to run it

Hydrofrac is driven from a test. Every protocol under test needs one, even when every property is supplied by the types: the test builds the flow, fixes cluster sizes, declares which nodes may crash and how many, and writes a body that drives inputs and timers. The body is not boilerplate — where it places the end of finite failures and how much recovery window it leaves decides whether a correct protocol certifies (§3; Part II §3). What the test does *not* have to do, on the common path, is say which collections to check or what to check about them. `Spec::all_labels` plus `Ldfi::search_all_labels` finds every IR node where the `EventualConsistency` label is *introduced* and every node where a labelled collection *exits* the labelled region, taps each, and runs one search per collection; the property is generated, only the harness is written (hydrofrac.md §"Using hydrofrac"). The label is a type parameter of the live collection — part of `Stream<T, L, B, O, R>`'s type, not an IR annotation — and the type system propagates it by its rules. It enters the type at exactly two kinds of operator: a network fan-out whose output location is `Cluster<_, N::ConsistencyGuarantee>` — `broadcast_closed` in `hydro_lang` (process→cluster and cluster→cluster, `live_collections/stream/networking.rs`) and, in `hydro_std`, `fan_out`/`fan_out_from_process` and their clients `broadcast_live`/`broadcast_live_from_process` (`ec_inference_demos/fan_out.rs`, `broadcast_live.rs`) — over a transport that grants it, which at `2f156b6335` means `TCP.fail_stop()`,

`TCP.lossy_delayed_forever()` and `UDP.lossy_delayed_forever()` (plain `lossy` grants `NoConsistency`, and `send`, `demux`, `round_robin` and the membership-snapshotting `broadcast` grant nothing; `networking/mod.rs`); or a programmer's `assert_has_consistency_of` with a proof — which is also what each fan-out is underneath, a `demux` followed by a library `manual_proof!`. The compiler records each node's resulting type in its IR metadata (`metadata().consistency`), so the search can walk the IR and pick out the nodes whose output carries the label while none of their inputs do (`sim::observe`; auto-labels spec §3). Those are the introducing nodes, and each is a place where the type system has made an assumption about faults that nothing in the program checks. Downstream of one, the label marks a region of the dataflow graph. Downstream operators inherit the label and are not listed. The collection exits the region at the first consumer that does not carry the label (`weaken_consistency`, a `batch` into an unlabelled tick) or at a program output; a send to another location is not an exit, the collection continues there under that transport's rule. An exit is where the program cashes the guarantee: the last collection to carry the label before its contents are consumed by code that no longer tracks consistency, and so the collection whose agreement across members the rest of the program actually relies on. Both are checked because they can disagree, and the disagreement is the point: reliable broadcast's single-hop `broadcast_closed` introduces `EventualConsistency` and is violated by the sender's crash mid-fan-out, while its delivered set — derived from that same label by the type system — certifies, because of the echo the types cannot see (hydrofrac.md §"Which collections are checked").



`reliable_broadcast_closed` as `Spec::all_labels` sees it (`molly-viz-samples/figures/reliable_broadcast_regions.dot`). The shaded region is the collections whose type carries `EventualConsistency`: the source's `broadcast_closed` (`:130`) and the members' echo `broadcast_closed` (`:156`) are its two introducing nodes, bold, where the transport's `fail_stop` guarantee puts the label into the type; `forward_ref`, `merge_unordered` and `unique` inherit it; and the delivered set leaves the region at the test's `weaken_consistency`, double-bordered, the exit. `all_labels` produces one verdict for each of the three — the two introducing nodes violated, the exit certified.

The simplest case first: one process broadcasting two items to a cluster of three over `fail_stop`, every node crashable. This is `ec_ldfi.rs::ec_broadcast_closed_all_labels` (added on the `hydrofrac-crash-budget` branch; the older `ec_broadcast_closed_fail_stop_breaks_under_a_sender_crash` checks the same collection through the hand-written form of Part II). The harness, trimmed to the lines that matter:

```
let mut flow = FlowBuilder::new();
let source = flow.process::<Source>();
let members = flow.cluster::<Member>();
let (send, items) = source.sim_input::<Item, TotalOrder, ExactlyOnce>();

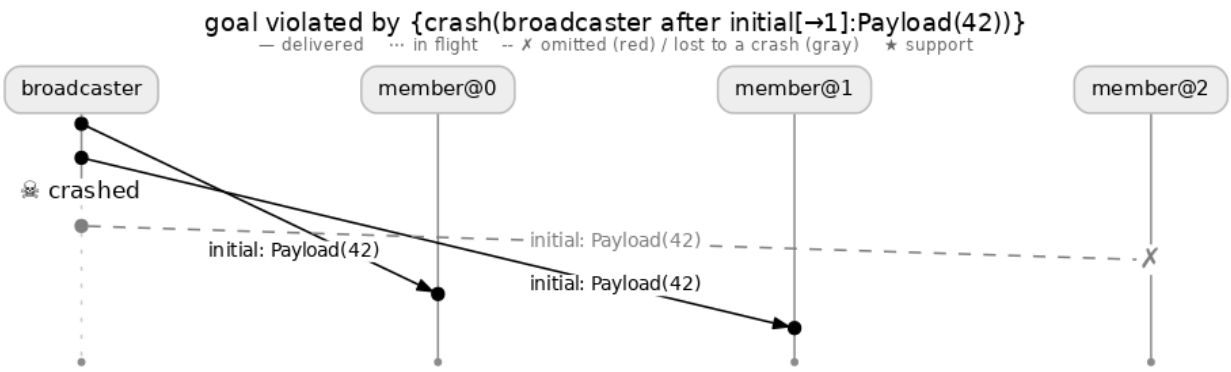
// broadcast_closed over fail_stop introduces EventualConsistency at the members.
```

```
let at_members = items.broadcast_closed(&members, TCP.fail_stop().name("bcast").bincode());
// Drop the label and make the stream a program output: the exit of the labelled region.
at_members.weaken_consistency().sim_cluster_output();

let spec = Spec::all_labels(&mut flow);
let compiled = flow.sim()
    .skip_consistency_assertions()
    .with_cluster_size(&members, 3)
    .with_provenance()
    .compiled();

let results = Ldfi::new()
    .crashable("source", &source)
    .crashable_cluster("member", &members, 0..3)
    .spec(&spec)
    .search_all_labels(&compiled, async |_run| {
        send.send(Item(1));
        send.send(Item(2));
        hydro_lang::sim::quiesce().await;
    });
println!("{results}");
```

Nothing is named: `all_labels` finds the one labelled collection — the members' side of the `broadcast_closed`, which is both the introducing node and, through `weaken_consistency`, the exit — and searches it. The verdict is **Violated** by the source's crash between its sends to two members, `crash(source after bcast[→1]:Item(2))`, in 5 hypotheses under the hand-written form (hydrofrac.md §"Results on existing Hydro code") [PENDING: counts from the committed `all_labels` run].



The failing run for plain broadcast (`molly-viz-samples/plain_broadcast`), found after 2 hypotheses and 8 schedules: one lane per node, time downward; the broadcaster delivers `initial: Payload(42)` to member 0 and member 1, then `☠ crashed` ends its lane after `initial[→1]`, and its third send to member 2 is drawn gray dashed to a `✗` — lost because the sender had crashed — so members 0 and 1 hold the fact and member 2 never does. A red dashed `✗` would be an omission on a lossy channel, a dashed `EFF` rule the end of finite failures, and `★` marks the goal's support in a witness diagram; none of the three appears here because the channel is `fail_stop` and the body sets no cut.

What the picture shows is the premise the type system made. The transport granted `EventualConsistency` on the reasoning that "a failed connection stops *all* future deliveries to that recipient, which models the recipient as having failed" (`hydro_lang/src/networking/mod.rs`, `FailStop`'s `ConsistencyGuarantee`) — a statement about recipients that is silent about the sender — and one crash of the sender between two sends of one fan-out leaves two members with the item and one without, with nothing to retransmit it. This is the standard EC violation, and every single-hop `broadcast_closed` in the corpus (§5) fails the same way.

Whether the failure matters depends on what the program builds on top of the hop, and that is what `all_labels` separates. The second test is the harness for `reliable_broadcast_closed` (`ec_demos_ldfi.rs::ec_demo_reliable_broadcast_all_labels`), and with `all_labels` it checks every `EventualConsistency` collection inside the protocol — the inner single-hop `broadcast_closed` the source issues, the echo the members issue, and the delivered set — in one run:

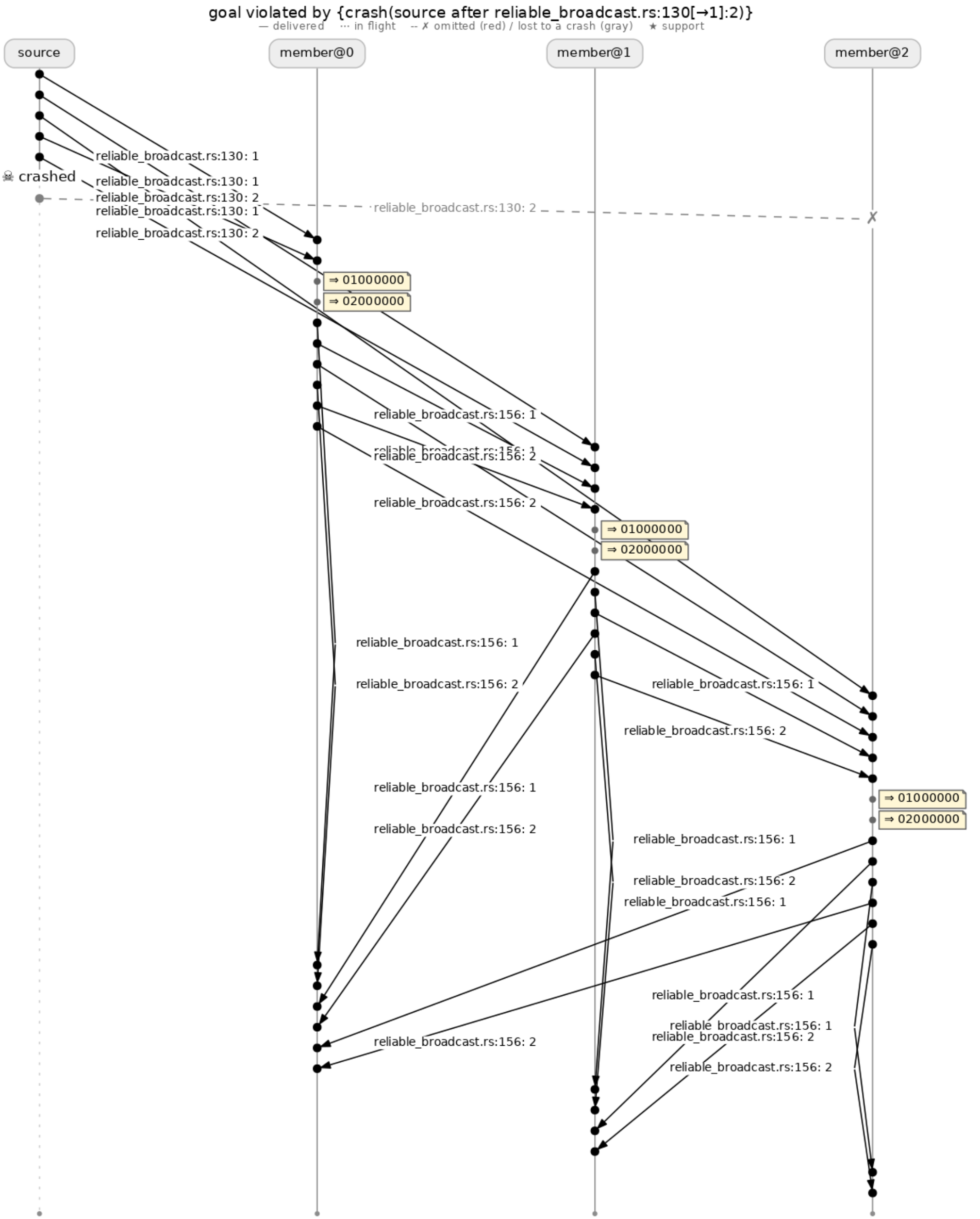
```
let mut flow = FlowBuilder::new();
let source = flow.process::<Source>();
let members = flow.cluster::<Member>();
let (send, items) = source.sim_input::<u32, TotalOrder, ExactlyOnce>();

let delivered = reliable_broadcast_closed(items, &members);   // labelled EventualConsistency
// Drop the label and make the stream a program output: this is the *exit*
// of the labelled region, and the collection all_labels reports as "delivered".
delivered.weaken_consistency().sim_cluster_output();

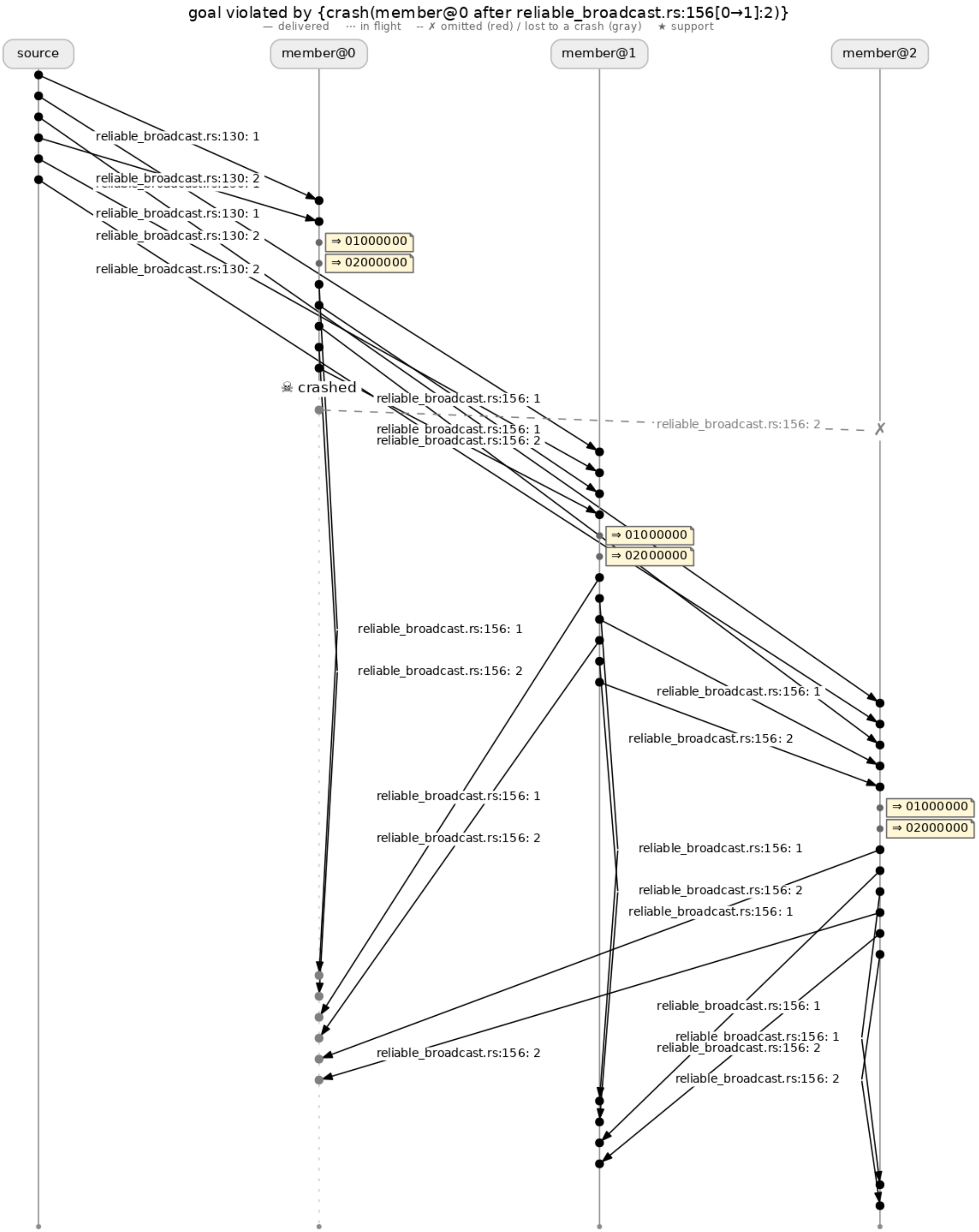
let spec = Spec::all_labels(&mut flow);
let compiled = flow.sim()
    .skip_consistency_assertions()
    .with_cluster_size(&members, 3)
    .with_provenance()
    .compiled();

let results = Ldfi::new()
    .level(Level::Precise)
    .crashable("source", &source)
    .crashable_cluster("member", &members, 0..3)
    .crash_budget(1)
    .spec(&spec)
    .search_all_labels(&compiled, async |_run| {
        send.send(1);
        send.send(2);
        hydro_lang::sim::quiesce().await;
    });
println!("{results}");
```

It prints one line per labelled collection: the source's single-hop broadcast violated by `crash(source after … [→1]:2)` in 5 hypotheses, the members' echo broadcast violated by an echoer's crash in 3, and the delivered set certified in 10 hypotheses under exhaustive schedules (hydrofrac.md §"Using hydrofrac"). The two violations are not defects. They are the premise the protocol exists to discharge: reliable broadcast builds a reliable delivered set out of an unreliable single-hop broadcast and an unreliable echo, and a result that reads "inner collections violated, delivered set certified" is the search showing the protocol doing exactly its job. The two inner failing runs:



`reliable_broadcast.rs:130`, the source's single hop (`molly-viz-samples/ec/ec_reliable_broadcast_inner_source_hop`): violated by `crash(source after reliable_broadcast.rs:130[→1]:2)` after 5 hypotheses and 5 schedules. The source delivers `1` and `2` to members 0 and 1, dies, and its `2` to member 2 is lost; member 2 receives `2` through the echoes on `:156` — its output ⇒ shows both items — but on this collection, the hop alone, member 2 holds `1` and members 0 and 1 hold `1` and `2`.



`reliable_broadcast.rs:156`, the members' echo, keyed by echoer (`molly-viz-samples/ec/ec_reliable_broadcast_inner_echo`): violated by `crash(member@0 after reliable_broadcast.rs:156[0→1]:2)` after 3 hypotheses and 3 schedules. Member 0 echoes 1 and 2 to member 1, dies, and its echo of 2 to member 2 is lost; member 1 holds (member 0, 2) and member 2 does not, while the merged delivered set is complete at both. The echo repairs the merged set, not the keyed record of who echoed.

The third line exists because of the `weaken_consistency().sim_cluster_output()` call: `weaken_consistency` strips the label from the type (`Stream<T, L::DropConsistency, ..>`), which is what makes the node an exit, and `sim_cluster_output` registers it as a per-member output of the simulated cluster (`SimClusterReceiver`). Without that line the delivered set is never consumed outside the labelled region and only the two introducing nodes are reported. `results.verdicts` holds one `LabelVerdict` per collection, `Searched(outcome)` or `NotChecked(reason)`; tests assert with `results.at("file.rs:149")`, `results.all_certified_or_not_checked()` or `results.counterexamples()`.

The generated specification, `Spec::eventual_consistency`, is a universal over a fixed fact domain: a failure-free probe collects every fact seen at any member into `FACTS`, and under every later hypothesis each fact of `FACTS` present at some correct member must be present at every correct member. A fact absent everywhere creates no obligation; a probe that yields no facts is `Vacuous` (hydrofrac.md §"What hydrofrac checks").

2. What the test has to set

Besides the flow and the body, an `all_labels` test sets four things on `Ldfi::new()`. The builder has other methods; they are tuning and measurement knobs and are listed in Part III §5.

Which nodes may crash, and how many. `.crashable("name", &process)` and `.crashable_cluster("name", &cluster, 0..n)` register the nodes a hypothesis may crash; `.crash_budget(k)` bounds how many crash in one hypothesis. The defaults — nothing crashable, budget 0 — are not a usable configuration for the EC check, only the absence of a declaration: the search is omissions-only until the test says otherwise. Why crashes are declared by the test while omissions are not, and what the right budget is, are in §3.

Which schedules. `.exploration(Exploration::Exhaustive)` (the default) explores every schedule the simulator can produce under each hypothesis; a certificate from it is a proof over that finite space. `Exploration::Fuzz { instances }` samples `instances` schedules per hypothesis; the result is then "no violation found", never a certificate (§4). The larger corpus runs — multi-Paxos, ISR, ABD — use `Fuzz { instances: 8 }` because exhaustive exploration of a leader-election run is out of reach; reliable broadcast and 2PC are exhaustive.

Which lineage. Compile the program `.with_provenance()`, as every example here does, and leave `.level()` at its default, `Level::Precise`. The level decides how the search works out which deliveries a member's output depended on — the information it needs to propose the next fault set — and `Precise` reads that from provenance the compiler recorded on each record. The only other level, `Level::Execution`, attributes every output to every input of the execution that produced it, which is coarser and makes the search try more hypotheses before it is done; both reach the same verdicts. If you forget `.with_provenance()`, the search still runs at `Precise`, recovering the same information by re-executing a node's code with a delivery removed, which is slower. Part III §1 is the measurement behind these two levels.

How long to search. `.max_hypotheses(n)`, default 256. The search ends in one of three ways — a counterexample, an infeasible solver (certificate), or this budget — and the third is reported as `Outcome::Budget`, which says nothing about the protocol (§4). The number is discussed with the other caps in Part III §5; what an EC user needs to know is that a `Budget` verdict means "raise it or shrink the test", and that in the corpus it was hit only by Raft and by noisy single-schedule Paxos runs.

Building with the `ldfi` feature needs cmake and libclang for the HiGHS solver bindings (commit d34bd78add).

3. What can fail and who can crash

A fault model answers three questions: which messages can be lost, which nodes can crash, and when faults may happen. In hydrofrac the program's transport declarations answer the first, the test's `crashable` registrations answer the second, and the test's body — the closure passed to `search_all_labels` that drives inputs and timers — answers the third.

What can fail. The fault point is a filter the simulator inserts before every `Network` send; every channel is a fault point by construction and nothing is registered. The transport type is the fault model. A `lossy_*` channel admits **omission**, identified by `(channel, sender, destination, wire bytes)`: every byte-identical copy on that channel is dropped while finite failures last, so a retransmission with the same payload is the same fault. A `fail_stop` channel admits no omission, only its **sender's crash** (ldfi.rs module docs §"Fault vocabulary"). A program whose channels are all `fail_stop` and whose test registers no crashable node has an empty fault universe and certifies after the failure-free run (ablation §"2PC … cr=0").

Who can crash. A crash is `crash(node after m)` or `crash(node before any send)`: the node performs every send up to and including `m`, every later send on every channel is dropped, and its external outputs are suppressed. Naming the point by a message rather than a send count keeps the hypothesis meaningful across runs whose send sequences differ, and lets one crash split the fan-out of a single broadcast — which is how every single-hop `broadcast_closed` in the corpus is violated. Only nodes registered with `.crashable` / `.crashable_cluster` may crash, and no more of them than `.crash_budget` allows. Why the test declares crashes while omissions need no declaration follows from where the information lives. A `lossy_*` transport *is* the program's statement that messages on that channel may be dropped. Nothing in the program says which processes may fail or how many at once — the types say a `fail_stop` sender may die, not whether the client, ZooKeeper or the coordinator is part of what is being tested — so that comes from the test (hydrofrac.md §"The fault model": "the program's transport declarations determine what can go wrong, while the test determines how much can go wrong and when faults must stop"). The corpus harnesses use the freedom deliberately: a `broadcast_closed` from a process has no premise other than its sender's survival, so with every node crashable every single-hop introducing node is violated in two hypotheses by the sender dying mid-fan-out (ISR's client, ABD's client, 2PC's coordinator, Paxos's proposer — hydrofrac.md §"Every label of ISR replication, two-phase commit, and ABD"). The harnesses hold those nodes correct so the search is about the protocol, then run them crashable separately to watch the premise fail.

How many. Every harness in the EC corpus allows one crash — "the searches below allow one crash", multi-Paxos "one crash allowed" (hydrofrac.md §"Every label of multi-Paxos") — and no source gives a reason: it is a convention, not a result. It can miss bugs. 2PC with cooperative termination certifies under one crash and breaks under two (history §9; Part II §4), which is exactly the shape a budget of 1 hides. For the generated property the ceiling is not `n − 1` but `n − 2` per cluster: with one correct member left, "every fact at some correct member is at every correct member" is trivially true, so the check needs at least two survivors to say anything. On the 3-member reliable broadcast `n − 2 = 1` and the harness is already at the ceiling; on multi-Paxos (3 acceptors, 3 learners, 1 proposer) it is not. The budget is global across every crashable node — `.crash_budget(k)` counts crashes, not crashes per cluster — so a per-cluster `n − 2` is not expressible today. The cost is real but bounded: echo reliable broadcast at `n = 3` goes from 10 hypotheses over 2,641 schedules at one crash to 32 over 7,271 at two, 50–190 s exhaustive (ablation §"Reliable broadcast" tables). The right budget for the generated check is therefore `n − 2` of the

cluster being checked; the convention of 1 in the corpus is correct only where `n = 3` makes the two coincide (reliable broadcast) and is too low elsewhere (multi-Paxos). Making `n - 2` the `all_labels` default, with a per-cluster budget in the API, is an open item (Part IV, open doors).

When. The body may mark an end of finite failures (`EFF`) once, with `run.end_of_finite_failures()`; after it no omission applies and no crash fires, and the rest of the run is failure-free recovery (ldfi.rs `Run::end_of_finite_failures`). The call is optional, and neither snippet in §1 makes it: without the call every send of the run is a fault candidate, which is right for a closed broadcast, since it has nothing to recover with. A protocol that converges by retrying needs the cut — with none, the adversary may drop every retransmission within the run — and where the body places it decides which messages the search can lose. Part II §3 has the full discussion and the open question about the form of the assumption.

The ISR harness's body (`isr_replication.rs:905–925`) is the shape to copy. The `run` it receives is the `Run` handle every body gets as its argument — the `_run` the earlier examples ignore:

```
async |run| {
    for w in 1..=writes {
        h.write_send.send(Write(w));           // the client's writes
    }
    hydro_lang::sim::quiesce().await;
    for round in 0..rounds {
        h.replica_timer.send_many((0..members).map(|m| (m, ())));   // replicas heartbeat
        hydro_lang::sim::quiesce().await;
        h.zk_timer.send(());                   // ZooKeeper recomputes the view
        hydro_lang::sim::quiesce().await;
        if round == 0 {
            run.end_of_finite_failures();      // faults may fall anywhere up to here
        }
    }
}
```

Everything before the call — the writes and the first round of replica and ZooKeeper timers — is where crashes and omissions may land; the remaining `rounds - 1` rounds run failure-free and are the protocol's chance to recover. Moving the call up to just after the writes would make the first round's traffic unloseable, and §5 shows what that changes.

4. Verdicts

Verdict	What it is worth	What it does not imply
Certified (`Outcome::Certificate`, `Grade::Generated`, `Exploration::Exhaustive`)	No admissible fault set, under any schedule, breaks the generated check within this test's topology, workload, horizon, crash budget and `EFF`.	Nothing about other inputs, cluster sizes or longer horizons; nothing about convergence beyond the horizon.
Violated (`Outcome::Counterexample`)	A concrete fault set and a concrete schedule under which some fact of `FACTS` is present at one correct member and absent at another at the end of the run. One failing run suffices; exhaustive exploration is not required.	That the protocol is wrong rather than the horizon too short; the trace must be read.

Verdict	What it is worth	What it does not imply
No violation under Fuzz	`instances` sampled schedules per hypothesis passed and no hitting set remained. Evidence.	A certificate; a violation in an unsampled schedule is missed.
Budget (`Outcome::Budget`)	The hypothesis cap was reached.	Anything about the protocol.
Vacuous (`Outcome::Vacuous`)	The failure-free probe collected no facts: the harness protects nothing.	Anything about the protocol; a statement about the body.
Not checked (`LabelVerdict::NotChecked(reason)`)	The collection is outside the generated specification's class: local inputs only, not a stream, inside a tick, excluded by the test.	Silence: the reason is printed, and nothing fails for a label the test does not assert on.

(hydrofrac.md §"Judgements"; ldfi.rs `Outcome`, `Grade`.) Every certificate on this path carries `Grade::Generated`. The grade follows the specification, not the search: `Spec::eventual_consistency` has by construction the form the hitting-set inference is sound for — a positive obligation over a fixed fact domain that no extra observed fact can undo (hydrofrac.md §"Why the specification's form matters"). A second grade, `Grade::HandWritten`, printed RELATIVE, exists for specifications the test writes itself; `all_labels` never produces it, and Part II §2 says what it is worth. A third, `Goal`, belongs to the superseded goal API. `Strategy::Blind` ends in `Outcome::Exhausted` rather than a certificate.

5. The EC corpus, example by example

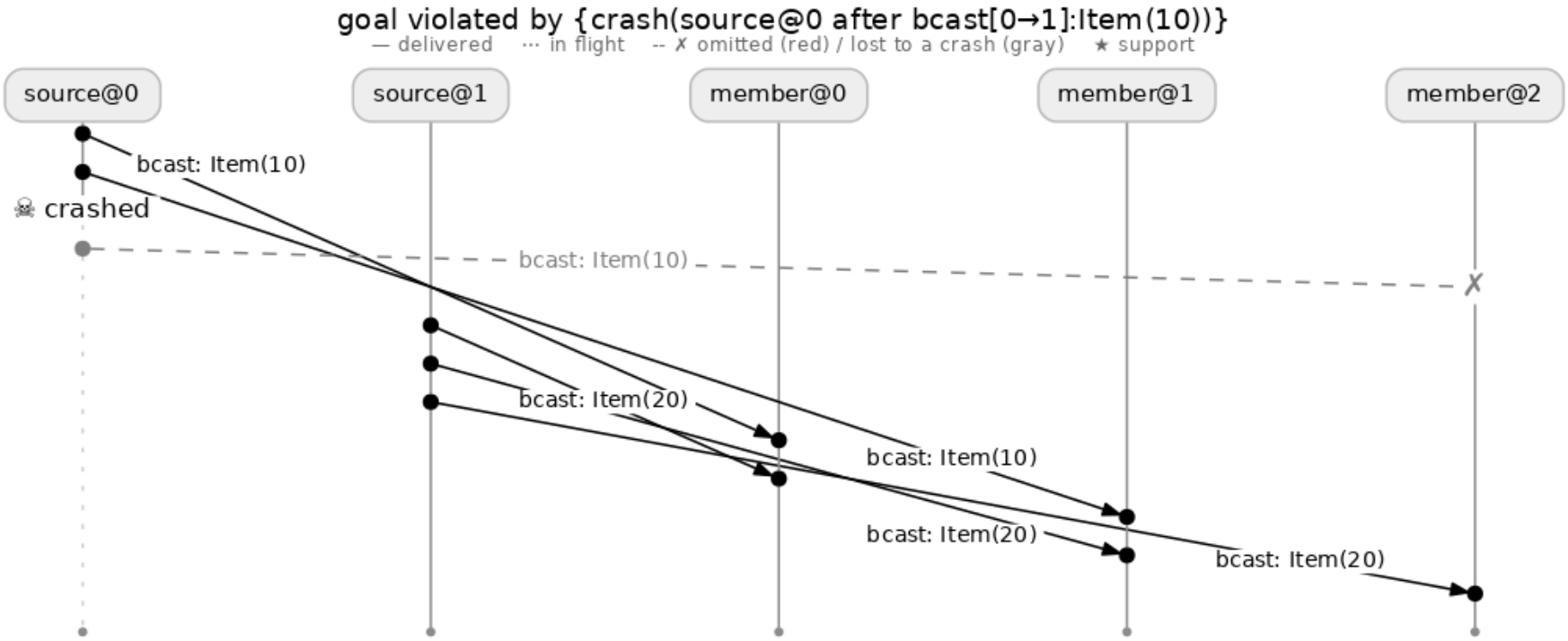
What the automatic search finds is standard and predictable, and that is the point. A single-hop `broadcast_closed` that introduces the label is violated by its sender's crash mid-fan-out, because the transport's grant assumes the sender finishes. A delivered set after an echo cycle certifies. The receiving side of the echo, a cluster → cluster broadcast keyed by echoer, is violated by an echoer's crash: the echo repairs the merged set, not the keyed record of who echoed (hydrofrac.md §"Every label of multi-Paxos"). None of this surprises anyone who has read the protocol. The value is that the verdict is per label, mechanical, and separates the transport's assumption from what the program delivers under it — the line the type system draws and cannot check. The rest of this section is the corpus one test at a time, after the two in §1: configuration, verdict, and the failing run where there is one. Hypothesis counts are taken from the gallery reports (`molly-viz-samples/ec/*/report.md`), with hydrofrac.md's tables, produced by different runs, as the cross-check; under `Fuzz` the count varies by one between runs (hydrofrac.md §"Results on existing Hydro code"), and every disagreement is noted. Counts include the probe.

`broadcast_closed`, cluster → cluster.
`ec_ldfi.rs::ec_broadcast_closed_cluster_to_cluster_breaks_under_a_source_crash`, body `cluster_to_cluster(1)`: a cluster of two sources, each broadcasting its own item, to three members over `fail_stop`; both clusters crashable, one crash, exhaustive. The receiving side of a cluster → cluster broadcast is keyed by the sending member, and the harness drops the key:

```
let sources = flow.cluster::<Source>();
let (send, items) = sources.sim_input::<Item, TotalOrder, ExactlyOnce>();
let at_members = items
    .broadcast_closed(&members, TCP.fail_stop().name("bcast").bincode())
    .values();
// ... .crashable_cluster("source", &sources, 0..SOURCES)
//     .crashable_cluster("member", &members, 0..MEMBERS)
//     .search(&compiled, async |_run| {
//         send.send(0, Item(10));
```

```
//         send.send(1, Item(20));
//         hydro_lang::sim::quiesce().await;
//     })
```

Violated, 5 hypotheses, `crash(source@0 after bcast[0→1]:Item(10))`, after 5 schedules (gallery report; hydrofrac.md's table agrees).



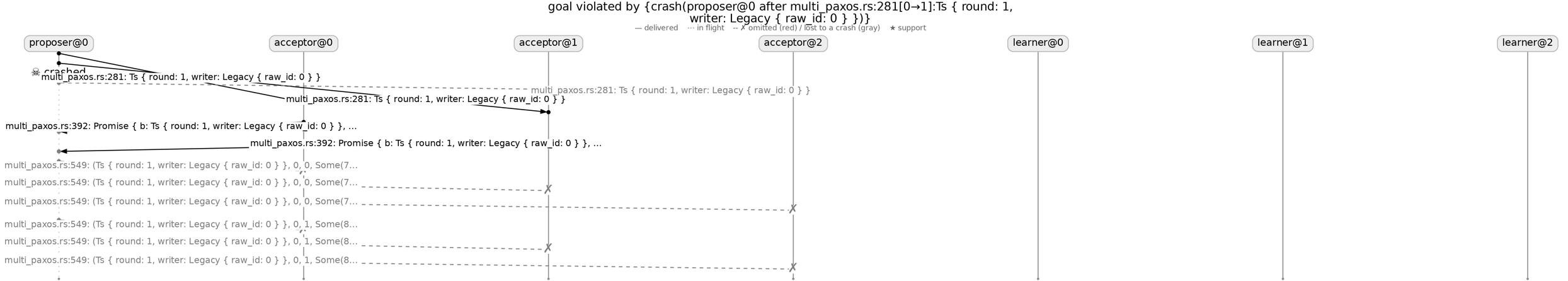
`molly-viz-samples/ec/ec_broadcast_closed_cluster_to_cluster`: `source@0` delivers `Item(10)` to members 0 and 1 and dies; its send to member 2 is gray dashed. `source@1` completes its fan-out of `Item(20)` to all three. Every member holds `Item(20)`; member 2 lacks `Item(10)`.

Each source member is its own single-hop broadcaster, so a cluster of `n` sources is `n` copies of the §1 premise; one crash in any of them is enough.

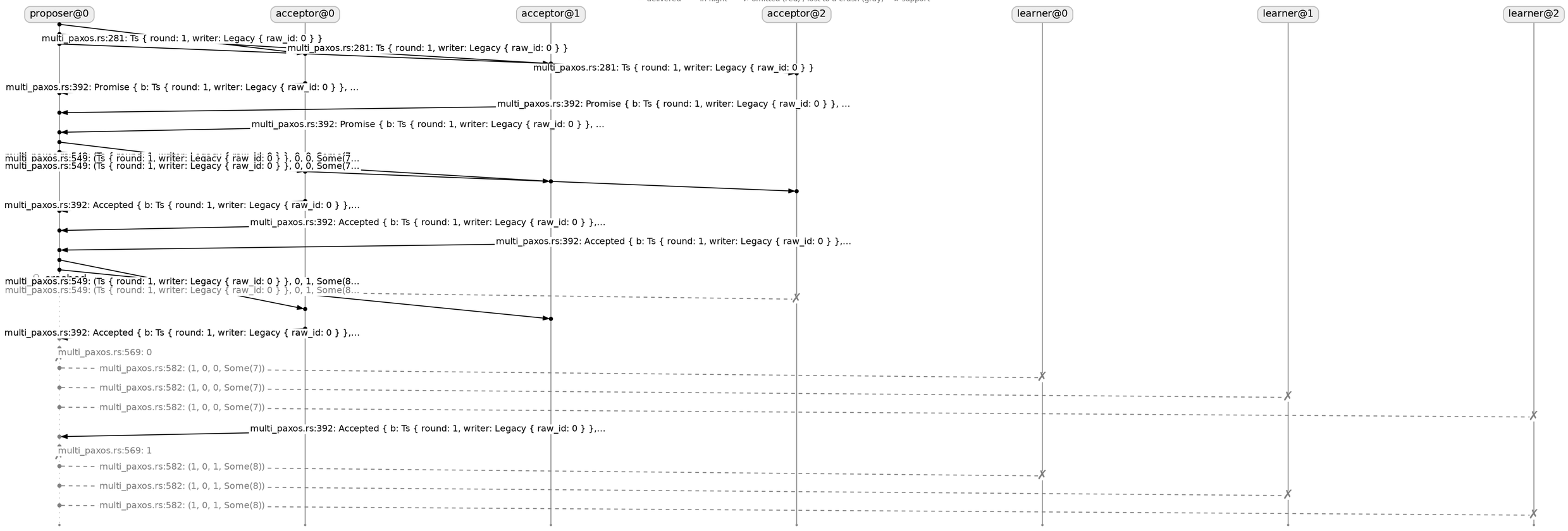
Multi-Paxos under `all_labels`. `ec_demos_ldfi.rs::ec_demo_multi_paxos_all_labels`: 1 proposer, 3 acceptors, 3 learners, all crashable, one crash, `Fuzz { instances: 8 }`. The body tells proposer 0 to lead round 1, quiesces, sends commands 7 and 8, and quiesces. The call is the §1 `all_labels` call with three crashable clusters and sampled schedules:

```
let outputs = multi_paxos(&acceptors, &learners, 2, leads, commands);
let _learned = outputs.learned.weaken_consistency().sim_cluster_output();
let spec = Spec::all_labels(&mut flow);
// ...
let results = Ldfi::new()
    .level(Level::Precise)
    .exploration(Exploration::Fuzz { instances: 8 })
    .crashable_cluster("proposer", &proposers, 0..1)
    .crashable_cluster("acceptor", &acceptors, 0..3)
    .crashable_cluster("learner", &learners, 0..3)
    .crash_budget(1)
    .spec(&spec)
    .search_all_labels(&compiled, async |_run| {
        lead.send(0, 1);
        hydro_lang::sim::quiesce().await;
        command.send(0, 7);
        command.send(0, 8);
        hydro_lang::sim::quiesce().await;
    });
```

It reports ten collections (line numbers into `hydro_std/src/ec_inference_demos/multi_paxos.rs`; hydrofrac.md §"Every label of multi-Paxos"). Five are violated, each by one crash: the proposer's `broadcast_closed` of prepares (`:281`, 5 hypotheses), of phase-2 accepts (`:549`, 8) and of chosen decrees to the learners (`:582`, gallery 10, hydrofrac.md 9), the `forward_ref` that carries `:549`'s entries into the acceptor's tick (`:286`, 8, by the same crash as `:549`), and the learners' echo keyed by echoer (`:591`, gallery 9, hydrofrac.md 10). Four report no violation under 8 sampled schedules per hypothesis: the chosen-slot broadcast among proposers (`:569`, 17–18) and its `.values()` (`:570`, 19), trivially, since the proposer cluster has one member; and the delivered `learned` set (`:588`, 25–26) and its `SpliceFact` map (`:608`, 25–26), because every learner that receives a decree echoes it to every learner and one crash leaves at least one echoer per decree. The `fold` into a `Singleton<SpliceState>` at `epoch_splice.rs:190` is not checked: not a stream. The two one-off count differences are the run-to-run variation hydrofrac.md records for `Fuzz`. The test asserts that `:588`'s exit certifies and that nothing hits the budget. One crash is the §3 convention, and here it is below the per-cluster ceiling: `n - 2` is 1 for the acceptors and 1 for the learners, and a global budget of 1 cannot crash one of each. The five failing runs:

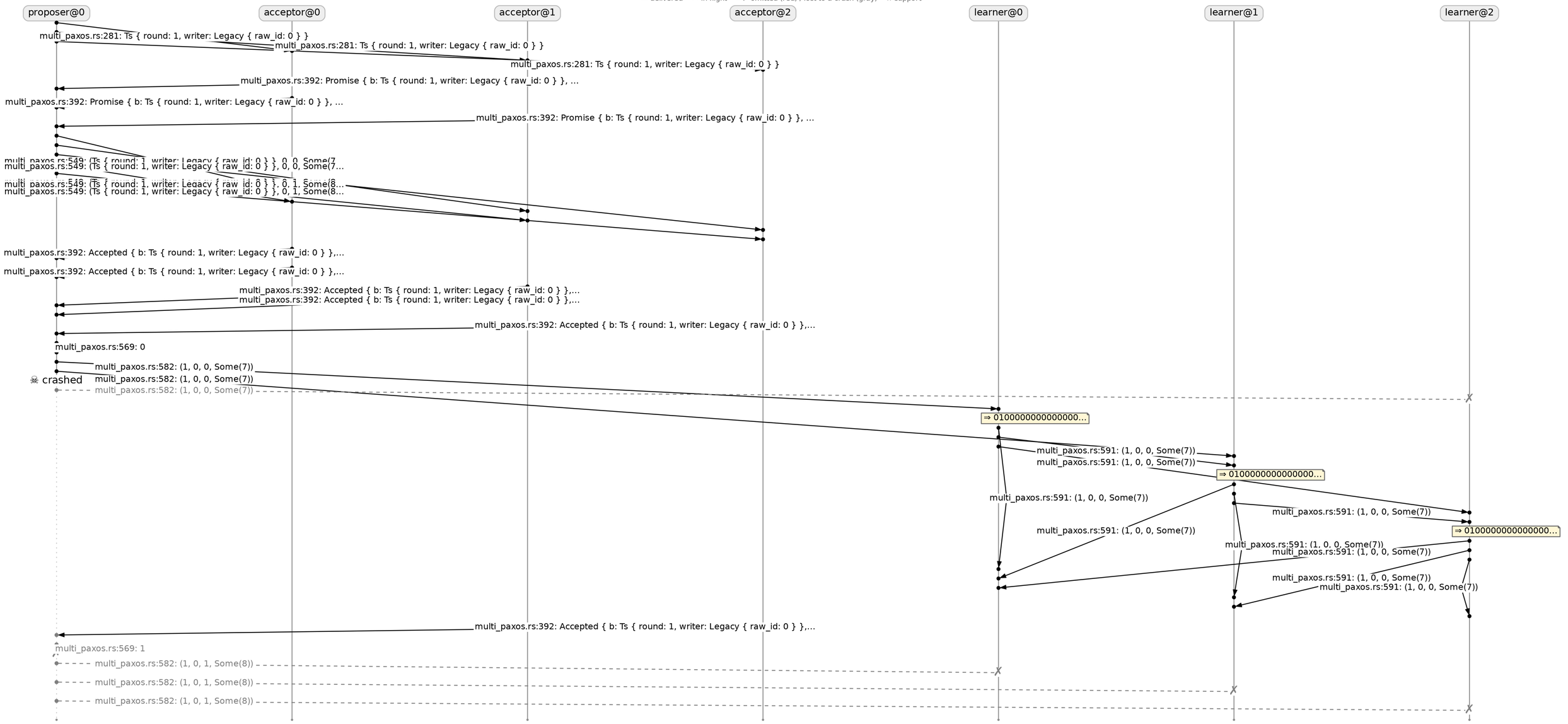


:281, prepares (`molly-viz-samples/ec/ec_multi_paxos_labels_281_prepares`): `crash(proposer@0 after multi_paxos.rs:281[0→1]:Ts { round: 1, writer: 0 })`, 5 hypotheses, 40 schedules. The round-1 prepare reaches acceptors 0 and 1, the proposer dies, acceptor 2's copy is lost; the two promises come back, and every phase-2 send below is gray dashed.



:549, phase-2 accepts (`molly-viz-samples/ec/ec_multi_paxos_labels_549_accepts`): `crash(proposer@0 after multi_paxos.rs:549[0→1]:(Ts { round: 1, writer: 0 }, 0, 1, Some(8)))`, 8 hypotheses, 64 schedules. The accept for slot 0 reaches all three acceptors; the accept for slot 1 reaches acceptors 0 and 1, the proposer dies, and acceptor 2 never sees slot 1.

`:286`, the `forward_ref` carrying the accepts into the acceptor's tick batch (`molly-viz-samples/ec/ec_multi_paxos_labels_286_accepts_fwd`): the same crash and the same failing run as `:549`, so the diagram above is this label's counterexample too — 8 hypotheses, 64 schedules. Acceptor 2 receives slot 0 and not slot 1. Nothing between the broadcast and the batch retransmits, so the exit inherits the hop's verdict.



:582, chosen decrees to the learners (`molly-viz-samples/ec/ec_multi_paxos_labels_582_learner_hop`): `crash(proposer@0 after multi_paxos.rs:582[0→1]:(1, 0, 0, Some(7)))`, 10 hypotheses, 80 schedules. Slot 0's decree reaches learners 0 and 1, the proposer dies, learner 2's copy is lost, and slot 1's decree is lost to all three. The learners' echo on `:591` carries slot 0 to learner 2 anyway — its ⇒ shows it — but on the hop alone learner 2 holds nothing. This is the premise the echo repairs.

goal violated by {crash(learner@0 after multi_paxos.rs:591[0→1]:(1, 0, 0, Some(7)))}

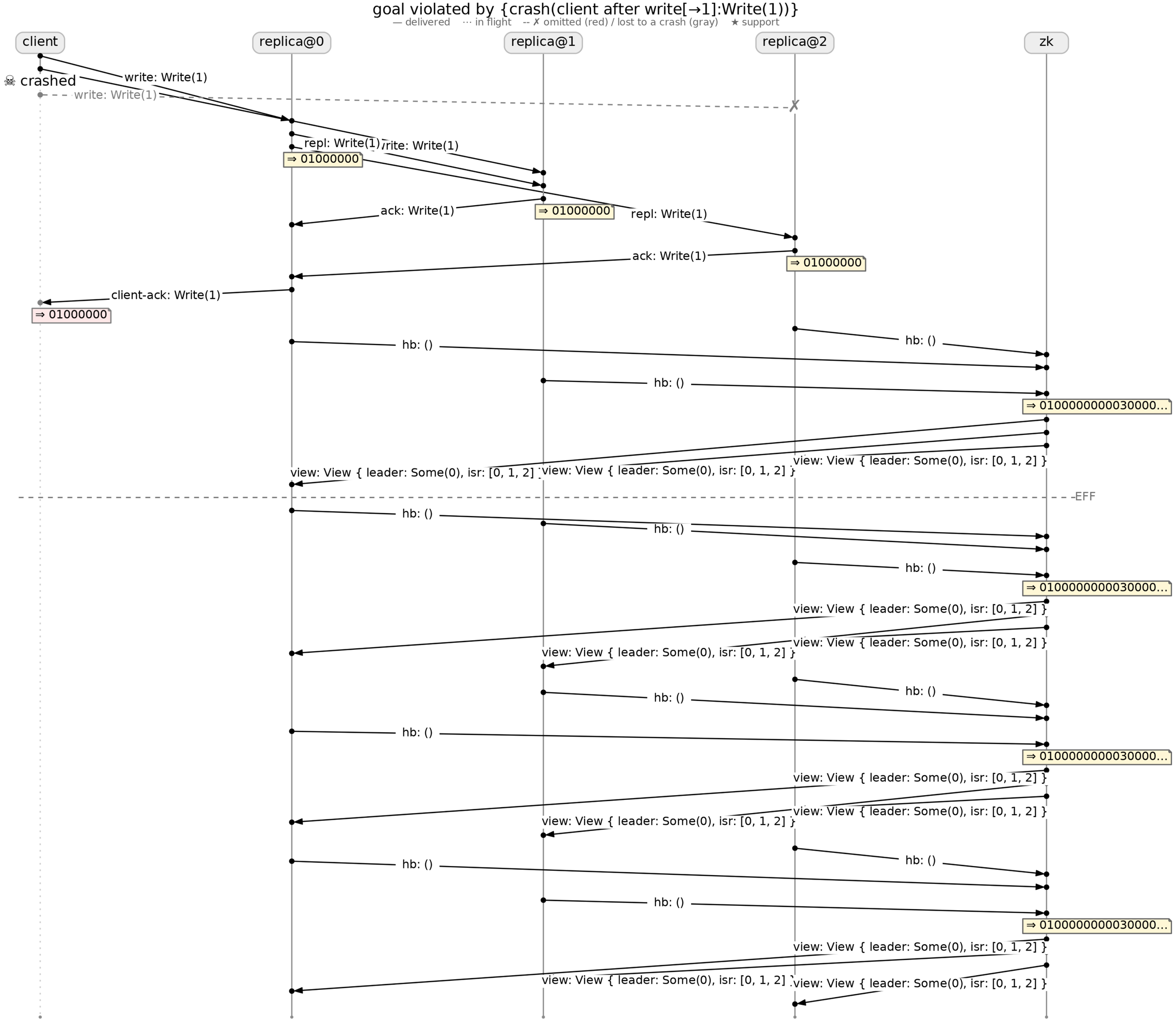
— delivered ⋯ in flight -- ✗ omitted (red) / lost to a crash (gray) ★ support

proposer@0 | acceptor@0 | acceptor@1 | acceptor@2 | learner@0 | learner@1 | learner@2

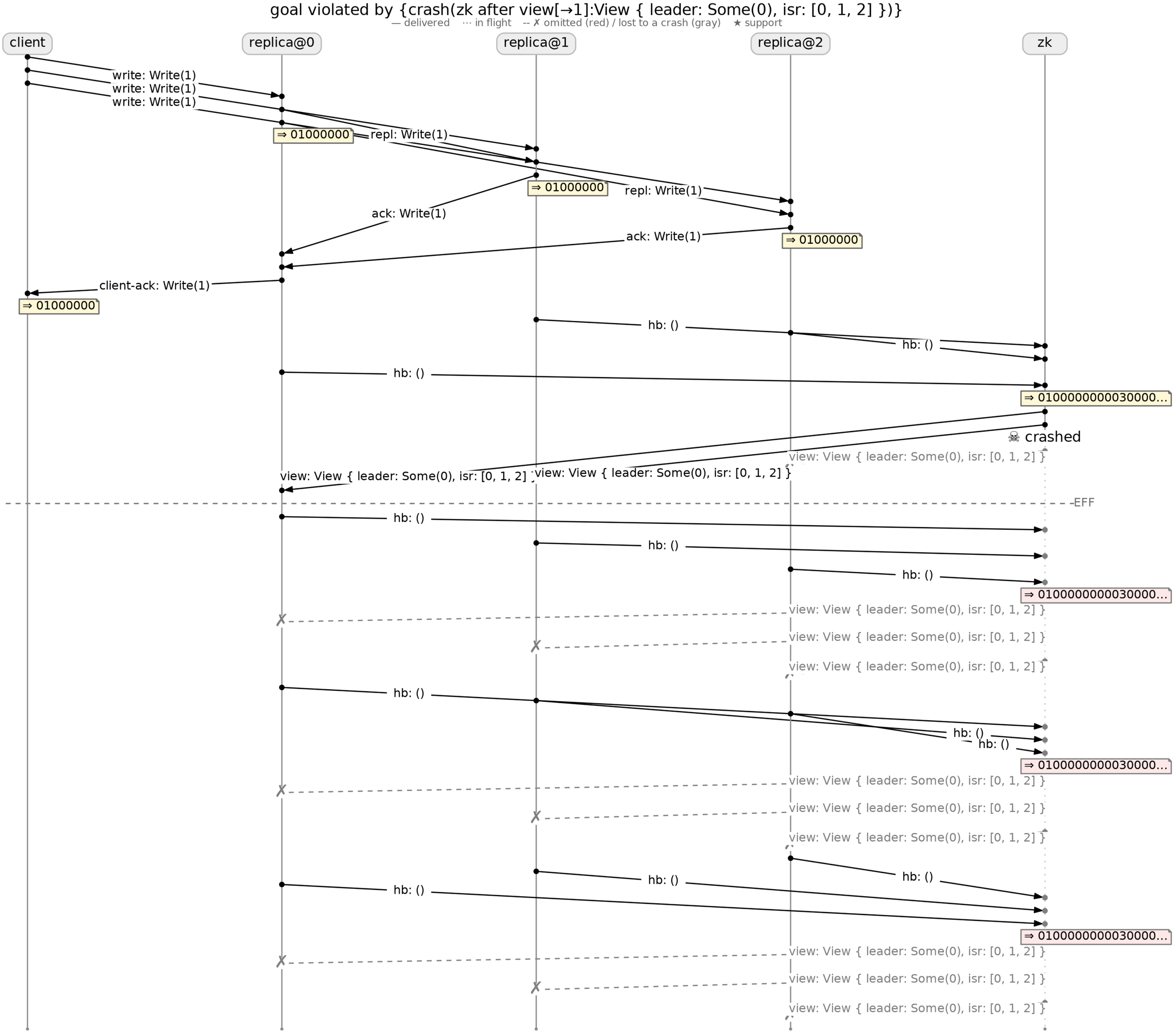
multi_paxos.rs:281: Ts { round: 1, writer: Legacy { raw_id: 0 } }
multi_paxos.rs:281: Ts { round: 1, writer: Legacy { raw_id: 0 } }
multi_paxos.rs:281: Ts { round: 1, writer: Legacy { raw_id: 0 } }
multi_paxos.rs:392: Promise { b: Ts { round: 1, writer: Legacy { raw_id: 0 } }, ...
multi_paxos.rs:392: Promise { b: Ts { round: 1, writer: Legacy { raw_id: 0 } }, ...
multi_paxos.rs:392: Promise { b: Ts { round: 1, writer: Legacy { raw_id: 0 } }, ...
multi_paxos.rs:549: (Ts { round: 1, writer: Legacy { raw_id: 0 } }, 0, 0, Some(7...
multi_paxos.rs:549: (Ts { round: 1, writer: Legacy { raw_id: 0 } }, 0, 0, Some(7...
multi_paxos.rs:549: (Ts { round: 1, writer: Legacy { raw_id: 0 } }, 0, 1, Some(8...
multi_paxos.rs:549: (Ts { round: 1, writer: Legacy { raw_id: 0 } }, 0, 1, Some(8...
multi_paxos.rs:392: Accepted { b: Ts { round: 1, writer: Legacy { raw_id: 0 } },...
multi_paxos.rs:392: Accepted { b: Ts { round: 1, writer: Legacy { raw_id: 0 } },...
multi_paxos.rs:392: Accepted { b: Ts { round: 1, writer: Legacy { raw_id: 0 } },...
multi_paxos.rs:392: Accepted { b: Ts { round: 1, writer: Legacy { raw_id: 0 } },...
multi_paxos.rs:392: Accepted { b: Ts { round: 1, writer: Legacy { raw_id: 0 } },...
multi_paxos.rs:569: 1
multi_paxos.rs:582: (1, 0, 1, Some(8))
multi_paxos.rs:582: (1, 0, 1, Some(8))
multi_paxos.rs:582: (1, 0, 1, Some(8))
⇒ 0100000000000000…
multi_paxos.rs:591: (1, 0, 1, Some(8))
multi_paxos.rs:591: (1, 0, 1, Some(8))
⇒ 0100000000000000…
multi_paxos.rs:591: (1, 0, 1, Some(8))
multi_paxos.rs:591: (1, 0, 1, Some(8))
⇒ 0100000000000000…
multi_paxos.rs:591: (1, 0, 1, Some(8))
multi_paxos.rs:591: (1, 0, 1, Some(8))
multi_paxos.rs:591: (1, 0, 1, Some(8))
multi_paxos.rs:591: (1, 0, 1, Some(8))
multi_paxos.rs:591: (1, 0, 1, Some(8))
multi_paxos.rs:591: (1, 0, 1, Some(8))
⇒ 0100000000000000…
multi_paxos.rs:591: (1, 0, 1, Some(8))
multi_paxos.rs:591: (1, 0, 1, Some(8))
multi_paxos.rs:591: (1, 0, 1, Some(8))
multi_paxos.rs:591: (1, 0, 1, Some(8))
multi_paxos.rs:392: Accepted { b: Ts { round: 1, writer: Legacy { raw_id: 0 } },...
multi_paxos.rs:569: 0
multi_paxos.rs:582: (1, 0, 0, Some(7))
multi_paxos.rs:582: (1, 0, 0, Some(7))
multi_paxos.rs:582: (1, 0, 0, Some(7))
⇒ 0100000000000000…
crashed
multi_paxos.rs:591: (1, 0, 0, Some(7))
multi_paxos.rs:591: (1, 0, 0, Some(7))
⇒ 0100000000000000…
multi_paxos.rs:591: (1, 0, 0, Some(7))
multi_paxos.rs:591: (1, 0, 0, Some(7))
⇒ 0100000000000000…
multi_paxos.rs:591: (1, 0, 0, Some(7))
multi_paxos.rs:591: (1, 0, 0, Some(7))
multi_paxos.rs:591: (1, 0, 0, Some(7))
multi_paxos.rs:591: (1, 0, 0, Some(7))
multi_paxos.rs:591: (1, 0, 0, Some(7))
multi_paxos.rs:591: (1, 0, 0, Some(7))

:591, the learners' echo keyed by echoer (`molly-viz-samples/ec/ec_multi_paxos_labels_591_learner_echo`): `crash(learner@0 after multi_paxos.rs:591[0→1]:(1, 0, 0, Some(7)))`, 9 hypotheses, 72 schedules. Learner 0 echoes slot 0 to learner 1 and dies before echoing it to learner 2; under learner 0's key that echo is at learner 1 and not at learner 2, while the merged `learned` set is complete at both.

ISR replication under `all_labels`.
`isr_replication.rs::ldfi_all_labels_break_under_a_sender_crash`, `compile_all_labels(false, Encoding::Dataflow, DEFAULT_SHAPE, true, 8)`: 3 replicas, 4 rounds, 1 write; client, ZooKeeper and replicas crashable; one crash; `Fuzz { instances: 8 }`. The body sends the write and quiesces, then runs four rounds of replica timers and a ZK timer, and calls `run.end_of_finite_failures()` at the end of round 0. The protocol labels three collections (line numbers into `isr_replication.rs`): the client's write broadcast (`:343`, introduced and delivered), the replication `forward_ref` at the replicas (`:360`, delivered) and ZK's view broadcast (`:564`, introduced). **Violated**: `:343` in 2 hypotheses by `crash(client after write[→1]:Write(1))` (gallery: 2 hypotheses, 16 schedules), and `:564` and `:360` in 8 each by `crash(zk after view[→1]:View { leader: Some(0), isr: [0, 1, 2] })` (gallery: 8 hypotheses, 64 schedules each; hydrofrac.md §"Every label of ISR replication, two-phase commit, and ABD" agrees). The companion test `ldfi_all_labels_with_the_harness_fault_model`, with the client and ZK held correct, reports no violation on all three in 4 hypotheses each: no crashable node sends on them. And with `EFF` right after the write, as the first version of the body had it, `:564` and `:360` report no violation in 5 hypotheses, because every ZK send lies after the cut and no ZK crash point is admissible (hydrofrac.md, same table); Part II §3 is the discussion of where the cut belongs.



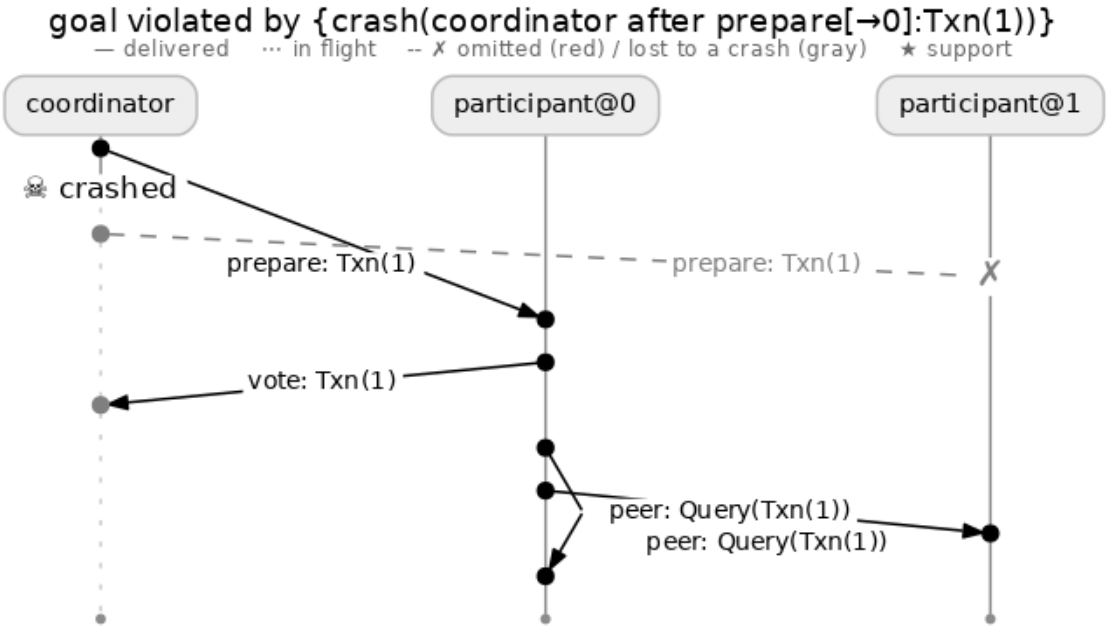
`:343`, the write broadcast (`molly-viz-samples/ec/ec_isr_all_labels_343_write`): the client's `Write(1)` reaches replicas 0 and 1, the client dies, replica 2's copy is lost. The leader's `repl` carries `Write(1)` to replica 2 anyway and the `client-ack` is suppressed at the dead client; the rest of the run — heartbeats, views, the `EFF` rule — is failure-free. On the write broadcast itself, replica 2 holds nothing.



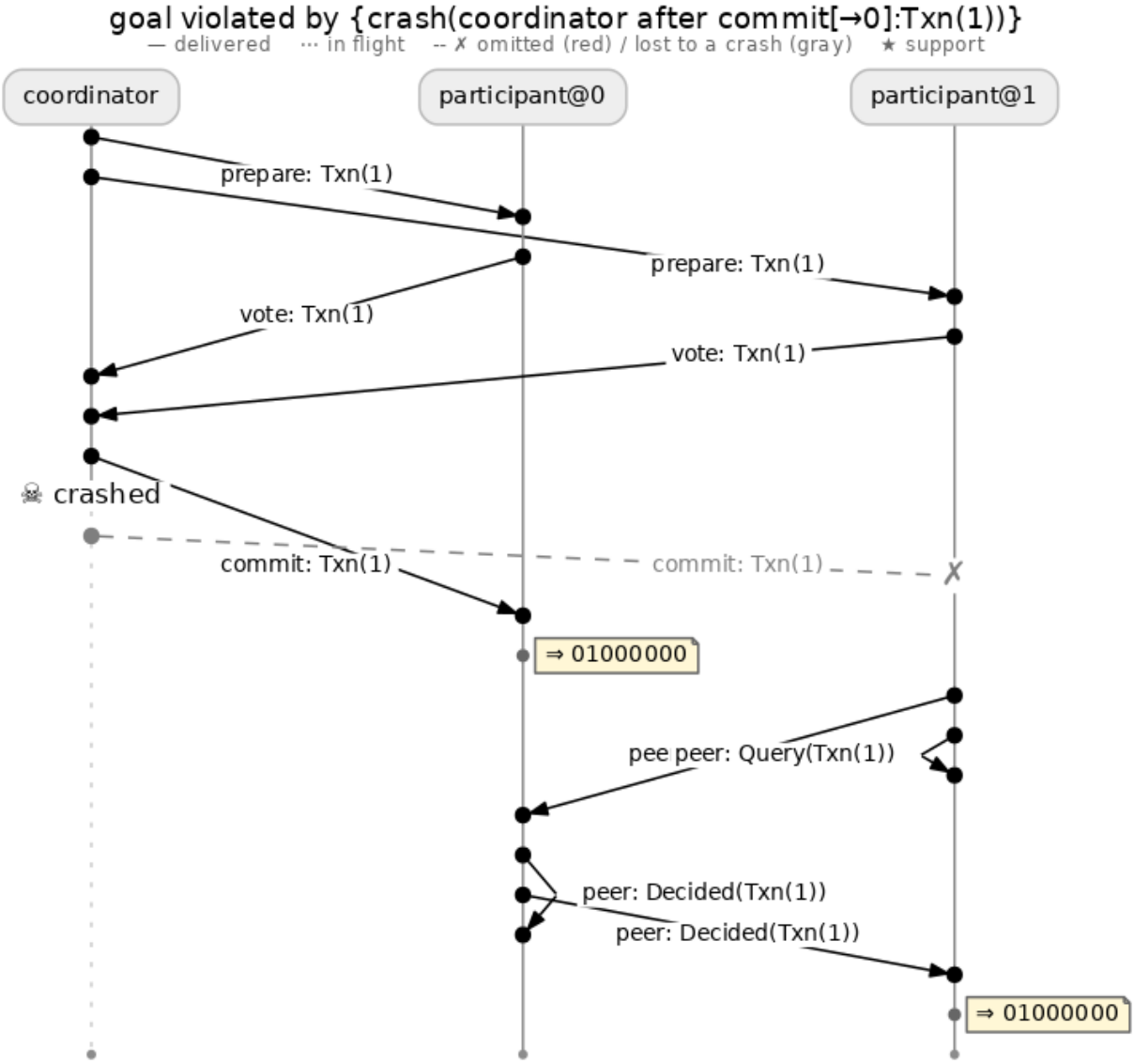
`:564`, the view broadcast (`molly-viz-samples/ec/ec_isr_all_labels_564_view`): the first view reaches replicas 1 and 0, ZK dies, replica 2's copy is lost; the `EFF` rule follows, and every view ZK would have sent in the three later rounds is gray dashed because its sender is dead. Replicas 0 and 1 hold a view, replica 2 none.

`:360`, the replication `forward_ref` at the replicas (`molly-viz-samples/ec/ec_isr_all_labels_360_replication_fwd`): the same ZooKeeper crash and the same failing run as `:564`, so the diagram above is this label's counterexample too — 8 hypotheses, 64 schedules. The view feeds the replication cycle, and the difference in views carries through to it (hydrofrac.md §"Every label of ISR replication, two-phase commit, and ABD").

Two-phase commit under `all_labels`.
`two_pc_ldfi.rs::ldfi_all_labels_with_the_harness_fault_model`, `compile_all_labels(2, true, true, 8)`: two participants, one transaction, cooperative termination, coordinator and participants crashable, one crash, exhaustive. Four collections (line numbers into `two_pc_ldfi.rs`): the coordinator's prepare broadcast (`:86`) and commit broadcast (`:92`), each introduced and delivered, and the participants' peer channel (`:98` `forward_ref`, `:161` broadcast). **Violated**: `:86` in 2 hypotheses by `crash(coordinator after prepare[→0]:Txn(1))` (gallery: 2 hypotheses, 20 schedules) and `:92` in 2 by `crash(coordinator after commit[→0]:Txn(1))` (gallery: 2 hypotheses, 64 schedules; hydrofrac.md agrees on both). **Vacuous**: `:98` and `:161`, because the failure-free run carries no peer traffic — the termination timers fire after every decision is known, so no participant queries a peer — and a collection with no facts gives the specification nothing to protect. The companion `ldfi_all_labels_with_the_coordinator_correct` (`compile_all_labels(2, true, false, 8)`) certifies `:86` and `:92` in 3 and 5 hypotheses: a crashed participant is not required to hold anything, and the survivors receive everything (hydrofrac.md, same table).



`:86`, the prepare broadcast (`molly-viz-samples/ec/ec_two_pc_all_labels_86_prepare`): the prepare reaches participant 0, the coordinator dies, participant 1's copy is lost; participant 0 votes and queries its peer. Participant 1 never sees a prepare.



`:92`, the commit broadcast (`molly-viz-samples/ec/ec_two_pc_all_labels_92_commit`): both prepares and both votes complete, the commit reaches participant 0, the coordinator dies, participant 1's copy is lost. Participant 1 queries, participant 0 answers `Decided`, and participant 1 commits too — its ⇒ at the bottom. Cooperative termination repairs the decision; the commit hop is still a hop.

ABD, and the class boundary. ABD (`hydro_std/src/ec_inference_demos/abd.rs`, `all_labels_reports_the_register_as_not_a_stream`; 3 replicas, 1 client, one write, `Fuzz { instances: 8 }`, replicas and client crashable): the query broadcast (`:141`), the phase-2 `forward_ref` (`:150`), the writes filtered from it (`:161`) and the write broadcast (`:314`) are each one hop from the client and **violated** in 2 hypotheses each by the client's crash between two sends of its query or its write; the register `fold` at `:165` is a `Singleton`, **not checked**. `ec_all_labels_classes.rs::all_labels_reports_a_label_inside_a_tick` pins the other boundary: a `fail_stop` broadcast batched into the members' tick and labelled there by `assert_has_consistency_of` is reported `not checked: inside a tick`, while the broadcast before the tick is searched and violated by the source's crash, and the `all_ticks` after it is searched (hydrofrac.md §"Every label of ISR replication, two-phase commit, and ABD"). No figures for either.

The rest of the corpus, without figures: the remaining producers of the label and the set-valued demos, one crash allowed (hydrofrac.md §"Results on existing Hydro code"; `ec_ldfi.rs`, `ec_demos_ldfi.rs`):

Program	Result
`broadcast_live_from_process`	Violated, 5 hypotheses: a source crash interrupts the one-hop broadcast
`leader_merge_broadcast`	Violated, 2: a leader crash mid-fan-out; the demo documents leader survival as an assumption
Echo reliable broadcast (`hydro_test`); `reliable_broadcast_closed`, `_live`	Certified, 10 hypotheses each, exhaustive
`uniform_reliable_broadcast_closed`, thresholds 1 and 2	No violation, 28 hypotheses, 8 sampled schedules
`g_set_gossip`, `_live`	No violation, 13, 8 sampled schedules
`multi_paxos` learners' `learned`, by hand (`Spec::new`), proposer held correct	No violation, 24–25, 8 sampled schedules; the `all_labels` run of the same collection above takes 25–26 because the observer carries every label's facts

What the corpus exposes is a gap in the transport-level justifications for the label. Treating a failed connection as a failed recipient does not account for a sender that dies halfway through a fan-out; treating a dropped message as indefinitely delayed does not make the surviving recipients converge when nothing retransmits. A transport label alone does not establish eventual consistency for the program using it — the echo, the gossip, the cooperative termination does — and the per-label verdicts are the mechanical record of where each program's labelled collections fall on that line (hydrofrac.md §"Results on existing Hydro code").

6. Reading a verdict: the visualizations

A violation you cannot read is not useful, so `hydro_lang::sim::viz` (feature `ldfi`) draws Molly's two zoom levels and one more (molly-viz.md).

Zoom 0 and 1: Lamport diagrams. A `Trace` is the serializable record of one run: nodes with their crash point, events in global scheduling order, messages in send order with fate (`Delivered | Omitted | Crashed`), and the `EFF` cut. It keeps no payload bytes; a message matches a `Dependency` by `(channel_id, from, to, FNV-1a hash of the bytes)`. A failing schedule always retains its trace; `Ldfi::with_traces()` retains every schedule's, which the witness picture needs. `lamport::mermaid` emits a `sequenceDiagram` (`->>` delivered, `-x` omitted, `--x` lost to the sender's crash, 🔢 and `EFF` notes, ★ on highlighted messages); `lamport::dot` emits a pinned-position `neato` spacetime diagram (a lane per node, one rank per event on an invisible time spine, red dashed ✗ for omissions, gray dashed for crashed sends, ☠ `crashed` ending a lane, a dashed `EFF` rule; commit c3164dd837 added self-placed labels that overlap nothing across the fifteen gallery files). Every violated collection of the EC corpus has its `failing` diagram inline in this document (§1 and §5, from `molly-viz-samples/ec/`), and the four hand-written cases have theirs in Part II §4. `Outcome::viz()` returns a `Counterexample` with `lamport_*()` for the failing run, `witness()` for the last goal-held schedule with a trace, `support_lamport_*()` with `Trace::support(derivation)` highlighted, and `markdown()` for both as a report (commits 18fdd58cec, c3164dd837).

Zoom 2: the why-provenance graph. `Ldfi::with_provenance_graphs()` records a `ProvGraph` beside every derivation: `Goal`, `Output`, `Exec`, `Message`, `Alive`, `Base` and `Opaque` nodes; `Checked`, `Produced`, `Consumed`, `Carried`, `Derived`, `Counterfactual`, `Fallback` and `Alive` edges, each tagged `exec`, `record` or `replay`, so a `Precise` graph that hit an opaque record shows the `Opaque` node and the `Replay` edges beneath it. Recording never

changes a derivation; `ProvGraph::dependencies()` equals the flat `Derivation`, which `ldfi_viz::provenance_graphs_agree_with_derivations_at_every_level` checks at both levels. `collapsed()` elides executions for the compact goal → messages picture (commit 567798e8ea; molly-viz.md §"Zoom level 2+").

The gallery (`docs/design/molly-viz-samples/`) has five directories — `two_pc`, `isr_unclean_election`, `paxos_commit_quorum`, `reliable_delivery_gives_up`, `plain_broadcast` — each with `failing`, `witness_{execution,record}` and `provenance_{execution,record}` as `.dot` and `.png`, plus `report.md`, and an `ec/` directory with fourteen entries, one per violated collection of Part I, each with `failing` and `report.md` only. Regenerate with `MOLLY_VIZ_OUT=/tmp/gallery cargo test -p hydro_test gallery_ -- --test-threads=1`, then `neato -n2 -Tpng` for the Lamport files and `dot -Tpng` for the provenance graphs (gallery README). The ISR report shows the omitted `repl[0→1]`, the shrink, the acknowledgement, the 🔢, the `EFF` rule, and ZK's promotion of replica 1 beneath it.

Known limits. External outputs render as a hex prefix; nodes the fault model does not name appear as `Process(locNvM)`; operators a record traversed are not tracked, so the graph shows which messages a fact rests on, not the `join` that combined them; alternative derivations of a deduplicated record are not drawn (molly-viz.md §"Limits").

7. Non-coverage and cost

Hydrofrac runs only in the simulator; the filter is a simulator artefact, not an IR node, so nothing exists under `deploy` (history §10, §11.3). The fault vocabulary is omission and crash: no delay, no reordering (the schedule explorer's dimension), no partition primitive (omission sets express one), no Byzantine behaviour, no duplication (hydrofrac.md §"Limitations"). Embedded channels are not fault points (history §5.6). Scalar labelled collections (`Singleton`, `Optional`, `KeyedSingleton`), collections whose label is introduced inside a tick, and collections with local inputs only are reported and not checked; an element type that is not `Clone + Serialize` fails the generated crate's compilation and has to be excluded with `Spec::all_labels_except` (hydrofrac.md §"Which collections are checked"). Membership is static. A certificate is for one test, one topology, one workload, one horizon.

Cost, on an 8-core debug build: ISR unit searches take seconds to half a minute (ablation §"ISR replication"); the 2PC level grid of 144 searches takes 162 s (ablation §"2PC"); echo broadcast at `n=3`, two crashes, exhaustive, 52–76 s, while `n=4` exhaustive is 92,160 schedules per hypothesis (~5 min each) and runs under `fuzz 8` (ablation §"Reliable broadcast", §"Exhaustive feasibility probes"); the Paxos `Precise` certificate at 230 hypotheses took 116 s with one sampled schedule per hypothesis and 1143 s at 393 with four (ablation §"Final", §"Aggregate-local counterfactual witnesses"). The hand-written Raft and Paxos searches, half an hour to an hour each, are costed in Part II §4. Regression suites: `cargo test -p hydro_test ldfi -- --test-threads=1` (56 passed, 12 long certifications ignored, ~12 min) and `cargo test -p hydro_lang --features ldfi --lib sim` (233 passed) (commit 2f156b6335; ablation §"Regression suites").

Part II — Hand-written specifications

When the property you care about is not one the types express — agreement, durability, "the leader has every acknowledged write" — you write it yourself, as a Hydro flow, and `Ldfi::search` runs the same search over it. Everything in this part exists because hydrofrac is then checking a view it did not construct: the grade on the certificate, the soundness caveat behind the grade, and the weight that falls on where the body ends finite failures.

1. Writing one

`Spec::new(&mut flow)` adds an observer process to the program. You ship the streams the property mentions to that observer with `spec.observe(stream)` (from a process), `spec.observe_cluster(stream)` (from every member, tagged with the member) or `spec.observe_sequenced(stream)` (keeping arrival order). Over those streams you build two bounded `Optional`s in the observer's tick and register them as `spec.pre(..)` and `spec.post(..)`. `PRE` is the precondition that says the run is one the property speaks about; `POST` is the conclusion. A view holds if it is non-empty at the end of the run, as a Datalog goal holds at fixpoint (Part III §7). A run in which `PRE` is non-empty and `POST` is empty is a counterexample; a run in which `PRE` is empty says nothing; a `PRE` that never holds in the failure-free run is `Outcome::Vacuous`, a verdict about the harness. `spec.correct_members(&cluster, 0..n)` gives you the set of members the current hypothesis did not crash, for properties quantified over survivors. `Ldfi::search` runs one such specification; the search is the same as the generated path, only the views differ.

The ISR durability check (`hydro_test/src/cluster/isr_replication.rs`) is the pattern in full. `PRE`: the client's first write was acknowledged *and* a final leader exists (`acked.filter(w.0 == 1).first()` zipped with the max view's leader). `POST`: that leader is among the replicas that appended the write. If a crash lets a replica that never saw the write win the election, `PRE` holds and `POST` is empty, and the search reports the crash. The older goal API (`goal`, `goal_all_members`, `goal_witnessed`) still compiles but is superseded by `pre`/`post` (ldfi.rs module docs §"Goals").

2. What a hand-written certificate is worth: RELATIVE and monotonicity

A certificate from a `Spec::new` / `pre` / `post` specification is `Grade::HandWritten` and prints as **Certified, RELATIVE**. What it says: no hitting set remained; the recorded derivations persisted under every excluded fault set. What it does not say: that the inference was sound for this view. A hand-written `POST` falsified by an *extra* fact — agreement, uniqueness, a final value — can be missed (ldfi.rs `Grade`). The rest of this section is why.

The hitting-set inference — "a fault set that leaves this derivation intact cannot break the view" — is valid only for views monotone in the facts they observe. The generated specification has that form by construction: for a fixed fact and a fixed set of correct members, "every correct member holds this fact" is a positive obligation that another observed `(member, fact)` pair cannot undo, and fixing `FACTS` from the probe prevents a faulty run from creating a new obligation that no earlier lineage explains (hydrofrac.md §"Why the specification's form matters"). Hand-written agreement and durability are not monotone: agreement is the *absence* of a conflict, and durability of a write rests on the *final* leader view. A hypothesis can break either by adding a fact, removing nothing any clause names. Their certificates are graded `RELATIVE`, with no soundness claim for the inference (ldfi.rs `Grade`; history §10).

The concrete lesson was a false certificate. When the observer-level counterfactual landed, `spec_view` took only the replay-derived minimal sufficient set, and on ISR's durability `POST` — the final leader read off

`observe_sequenced(views).max()` — deleting views latest-first never changed the value, since the earlier view names the same leader. The sufficient set was the appended write and the *initial* view, with no heartbeat in its lineage; the breaking hypothesis (lose a replication, crash the leader after the acknowledgement, let ZK promote the stale follower) kills neither — it *adds* a later view. Six hypotheses, certificate, bug present. Record provenance, which folds every contribution of an aggregate into its value, kept the views and the heartbeats behind them, and `spec_view` now takes the **union** of the counterfactual and recorded derivations: sound where either is, minimal only where both agree (negation §4). The search finds the corpus's non-monotone bugs because the breaking hypothesis also displaces a recorded fact in `POST`'s derivation; a conflict that appeared without displacing one would not be proposed, and the grade says so.

3. Bounding failures: `EFF` and recovery windows

Since commit `4ddea33f19` the simulator enforces Molly's rule: every failure happens at a time `≤ EFF`, the end of finite failures. The body may mark that point once with `run.end_of_finite_failures()`; after it, omissions stop applying and no crash fires, and the rest of the run is failure-free recovery. The call is optional. Without it there is no cut — every send of the run is a fault candidate — which is what the snippet in Part I §1 does, correctly, because a closed broadcast has nothing to recover with (ldfi.rs `Run::end_of_finite_failures`). A body calls it when the protocol converges by retrying: over a lossy channel with no cut the adversary may drop every retransmission within the run, so no retry-based protocol could ever certify. Finite failures is the fairness assumption that makes the convergence question answerable at all (history §1), and the body has to fire the protocol's timers after the cut or it has not asked the question (history §9: four Raft bodies failed correct Raft this way before the fifth).

The rule is Molly's after two intermediate positions. The first version admitted crash points at any send; on Raft that "violated" durability on the *correct* protocol when the leader crashed in the heal phase after the body's last election timer had fired. A crash inside the convergence window needs a window of its own, which the body cannot supply without knowing where the crash fell (history §5.4). The search then confined *proposed* crash points to pre-`EFF` sends while the simulator still fired them anywhere — the clamp behind root cause 1 of Part III §2. Commit `4ddea33f19` closed the gap at the simulator: a crash point named by a pre-`EFF` send fires only if that send happens before `EFF` in the run at hand, and the harness places `EFF` where the failures it means to admit lie. One model, one window, both halves agreeing.

Where the cut falls decides which messages the search can lose, and the harness author decides where it falls. Two moves in the corpus show it. Commit `4ddea33f19` moved the ISR harness's cut past the first round so the acknowledgement lies inside the window in every world (Part III §2). The Paxos harness author moved its cut from before the new leader's first ballot to after it, which made the P1a/P1b exchange faultable, made the injected bug's counterexample independent of the schedule, grew the correct variant's certificate from 196 to 372 hypotheses, and exposed a production defect the old cut had hidden (§4; history §9.2). History §9.2's lesson: the harness that cannot lose a message cannot test the protocol's handling of its loss.

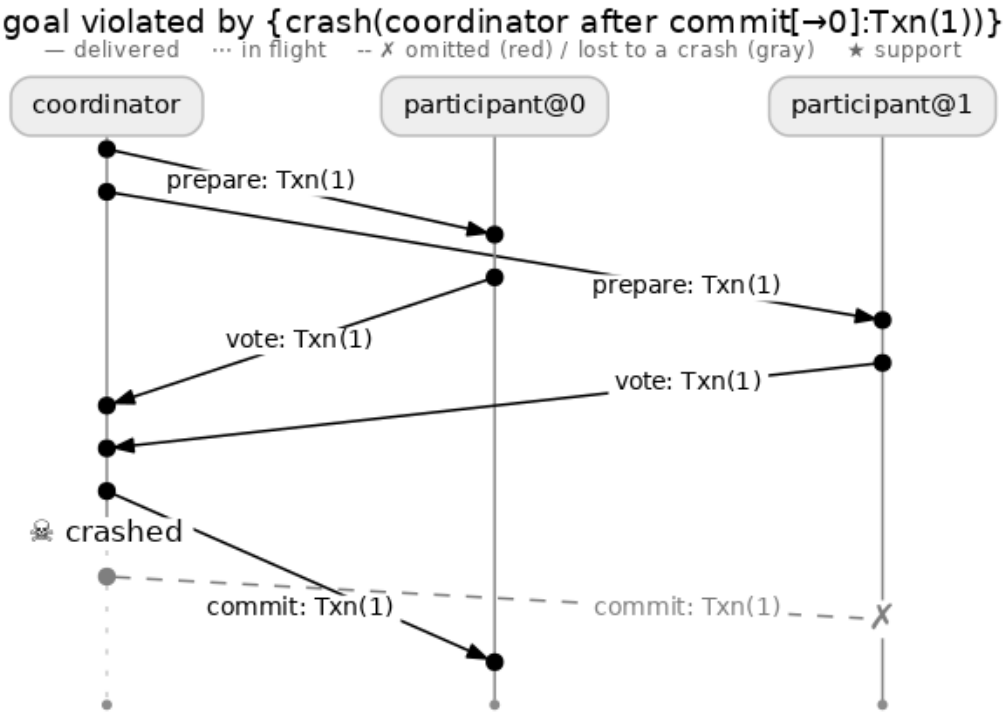
Whether this is the right *form* of the assumption is unresolved, and the document should not pretend otherwise. History §10, after round five, called `EFF` a leftover of Molly's logical-time search — "the horizon is what the body drives, and omission candidates are the sends the run observed … it should not shape the design further." The next day's commit `4ddea33f19` made it the fault model, for a stated reason that is about consistency, not about `EFF`: the soundness bug of Part III §2 was the solver and the simulator disagreeing about what a crash kills, and the fix chosen was to have both follow Molly's rule exactly. The commit does not say why the other fix — no cut, with the solver following the simulator's actual kill relation — was not taken. The one argument the sources do give for *some* cut is the Raft heal-phase case above: with crashes admissible anywhere, a crash in the heal phase after the body's last

timer "violated" durability on correct Raft, because a crash inside the convergence window needs a window of its own and the body cannot supply one without knowing where the crash fell (history §5.4). That justifies bounding the last failure; it does not say the bound has to be Molly's point on the body's timeline. [NEEDS AUTHOR REASONING: is `EFF` as a point on the body's timeline the intended form of the fairness assumption, or a stopgap until something like per-channel fair loss or the convergence witness of the open doors replaces it? History §10 and commit 4ddea33f19 point in opposite directions.]

4. What hand-written harnesses have found

Injected bugs, found as counterexamples. Each is a fault set the search proposed from the derivations it had seen, under which `PRE` held and `POST` did not.

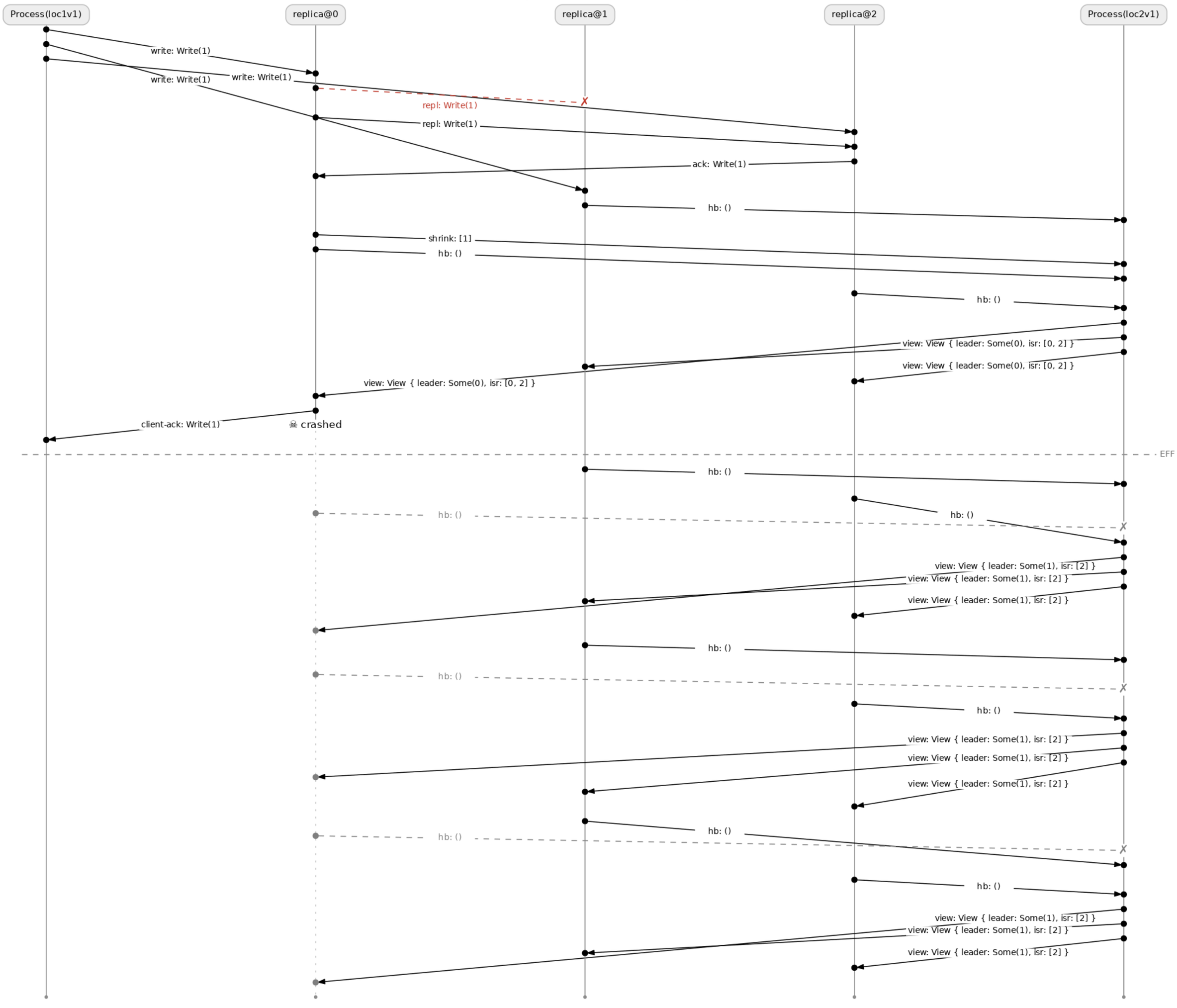
- Plain 2PC breaks under one coordinator crash between two `commit` sends, in 2 hypotheses at every size.



Plain 2PC (`molly-viz-samples/two_pc`), 2 hypotheses, 24 schedules: both participants prepare and vote, the coordinator sends `commit: Txn(1)` to participant 0, `☠ crashed` ends its lane, and the commit to participant 1 is lost gray to a `✗` — participant 0 commits and participant 1 blocks.

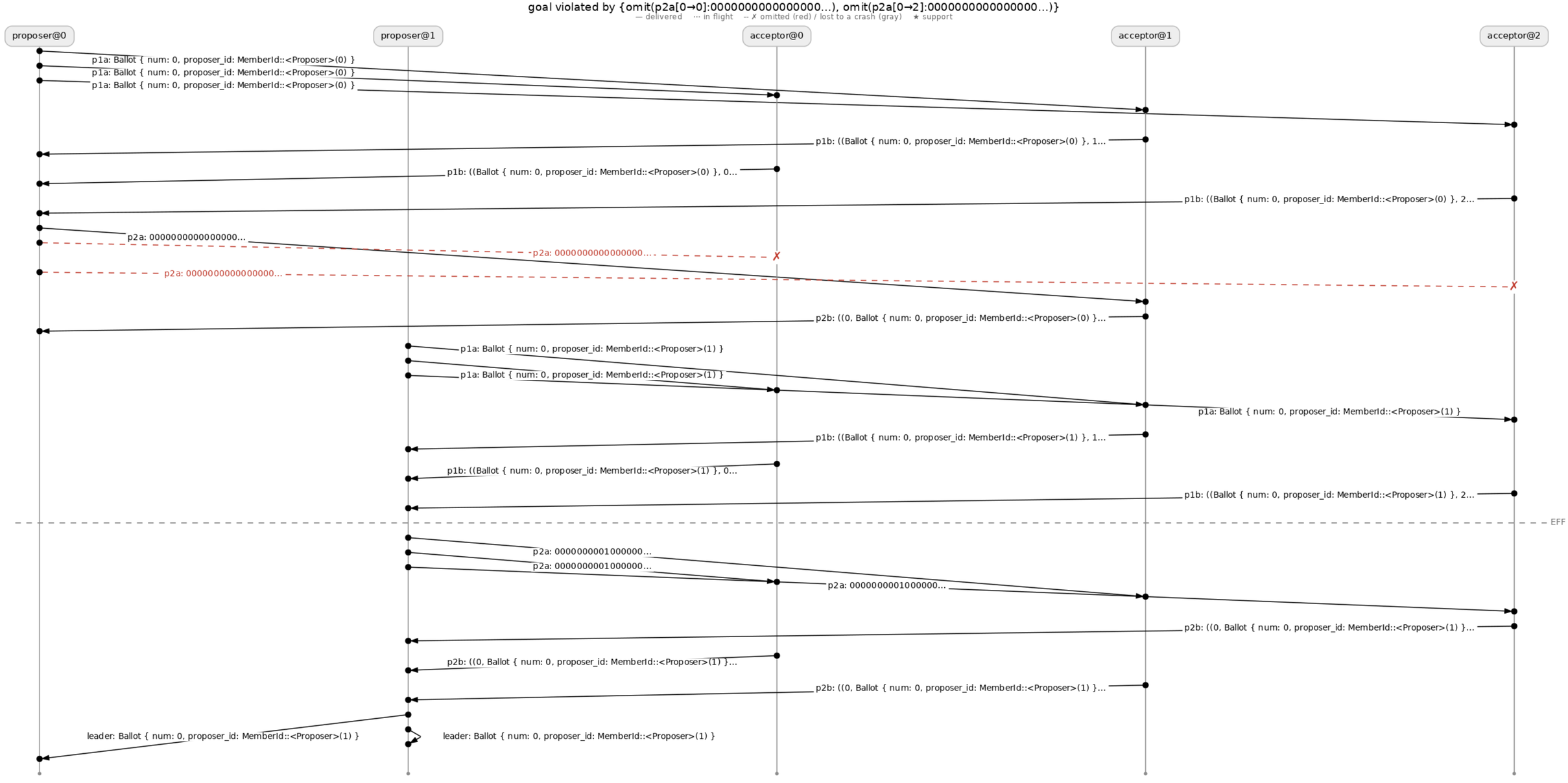
2PC with cooperative termination certifies under one crash in 9 / 13 / 18 / 23 hypotheses at `n = 2/2/3/4` (loads 1/ ≥ 4/ ≥ 4/ ≥ 4) at `Precise`, and breaks under two: the coordinator after one commit, then the participant that received it (ablation §"2PC"; hooks §"Prototype status"; history §9).

- ISR's unclean election — Kafka's bug, a replica that never saw an acknowledged write wins the election — is `{omit(repl[0→1]), crash(replica@0 after client-ack)}`, found in 9 (closure encoding) or 12 (dataflow) hypotheses at `Precise` after the soundness fix of Part III §2, and in 14 at `Execution` (ablation §"ISR replication").



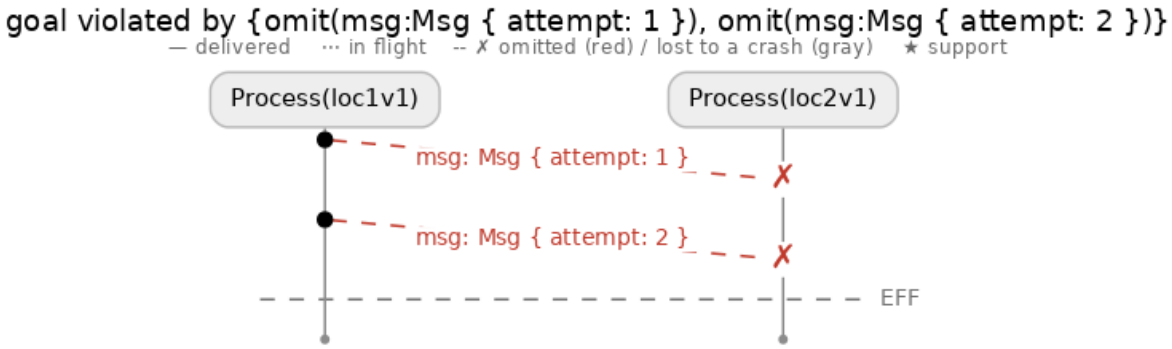
ISR (`molly-viz-samples/isr_unclean_election`), 14 hypotheses, 112 schedules: `repl[0→1]: Write(1)` is the red ✗, replica 0 shrinks replica 1 out of the ISR, is acknowledged by replica 2, answers the client with `client-ack`, and dies; below the `EFF` rule ZooKeeper's view promotes the stale replica 1 (`leader: Some(1)`).

- Raft's election restriction with index-only log comparison passes all thirteen of `raft.rs`'s behavioural tests and is found in 90 hypotheses with four omissions and no crash, about thirty minutes at four sampled schedules per hypothesis: member 0 wins term 1 and appends `r0`, lost on both links; member 1 wins term 2, appends `r1` at the same index and commits it, its replication to 0 lost twice; in the heal phase 0 campaigns with `(index 1, term 1)`, wins under the index-only check and overwrites the committed `r1` — a scenario I had not written down. Correct Raft certifies RELATIVE in 235 hypotheses, about an hour, under the same body (history §9; ablation §"Raft" for the 4–5 s per hypothesis). An earlier report of 17 hypotheses is withdrawn: it came from a body that staged the bug's scenario by hand — elected member 1, raced member 0's timer against a request, fired member 1's timer twice after the cut — so the search only had to find two omissions inside a scenario its author had arranged, and the Maelstrom comparison it fed credited lineage for conditions the body had set up. I wrote that body and reported the number without saying so. The 90 / 235 figures are from a generic body shaped only by what *correct* Raft needs to converge (history §9).
- Core Paxos with a commit quorum of `f` acknowledgements instead of `f + 1`, which every unit test passes: first found in 12–43 hypotheses across eight runs, but only under the schedule in which the new leader's P1b quorum missed acceptor 0, because the body's cut preceded the new leader's first ballot and that P1b was not a fault point. After the harness author moved the cut past the first ballot, the counterexample is the three omissions `{p2a(x)→1, p2a(x)→2, p1b(acceptor 0)→proposer 1}`, schedule-independent; correct Paxos certifies under that body in 372 hypotheses (history §9.2). The ablation's own Paxos harness finds the bug with two P2a omissions and certifies the correct variant at `Precise` in 230 hypotheses where `Execution` runs out at 256 (ablation §"Final: two levels").



The ablation harness's counterexample (`molly-viz-samples/paxos_commit_quorum`), 37 hypotheses, 148 schedules: proposer 0's `p2a` to acceptors 0 and 2 are the two red ✗, only acceptor 1 accepts `x` and its lone `p2b` is counted as a commit quorum (the injected bug); after the EFF rule proposer 1's P1b quorum misses `x` and it chooses `y` at slot 0.

- Compartmentalized Paxos (two proposers, two proxy leaders, a 2 × 2 acceptor grid, eight nodes) with a write quorum one acceptor short of a row: 3 hypotheses. The correct variant certifies in 568 (history §9.2).
- At the library level, `hydro_std::quorum::collect_quorum` between a coordinator and three retransmitting responders over lossy channels: the quorum claim certifies for a deduplicating caller in 28 hypotheses and breaks for the caller that feeds the library every reply in 5 — silence two responders and the third's retransmitted reply is counted twice (history §9).
- The LDFI paper's first example, a sender over a lossy link that retries twice and then gives up (`distributed::reliable_delivery`), is in the gallery for completeness rather than as a bug the corpus found: with the property *if the sender sends and not every copy is dropped, the receiver eventually has it*, the search loses both attempts before `EFF` in 3 hypotheses.



Reliable delivery (`molly-viz-samples/reliable_delivery_gives_up`), 3 hypotheses, 3 schedules: `msg: Msg { attempt: 1 }` and `attempt: 2` are both red ✗ omissions above the `EFF` rule; nothing is left to deliver once the network heals.

The three corrected Paxos certificates (372 for core Paxos, 301 for the replicated key-value store over it, 568 for compartmentalized Paxos) take about forty-five minutes together at four sampled schedules per hypothesis (history §9.2; hydrofrac.md §"Hand-written specifications").

Three defects in the production Paxos code that were not injected. `hydro_test/src/cluster/paxos.rs` had never run in the simulator — its leader liveness is on the wall clock — and once the harness made the timers inputs, three real defects came out. For each: what the bug was, and which instrument found it (history §9.2).

A message loop in the leader change. The values a new leader recommits were re-sent on every later tick, and each round of P2bs triggered the next tick, so once a recommit was needed the proposer and the acceptors exchanged the same P2as and P2bs forever. This was not a lineage counterexample: the run under hydrofrac's first faulty hypothesis never quiesced — the simulator hung — and the loop was found by reading why. Any fault injection that drove a leader change with a partially replicated slot would have found it; hydrofrac's contribution was the fault. The fix gates the recommit on the tick the proposer becomes leader.

Compartmentalized Paxos's read quorum counted rows, not columns. The leader election accepted any `rows` P1bs of the `rows × cols` acceptor grid, so a proposer could be elected on one row's P1bs, which does not intersect the other row — the write quorum of half the slots. Hydrofrac reported a violation of agreement on the *correct* variant in 8 hypotheses, but the failing schedule needed no fault: proposer 1's quorum was row 1 while `x` had been chosen on row 0, and only the P1b arrival order made it so. The simulator's schedule exploration found it; hydrofrac supplied the observer and the report that named the schedule. The fix has P1bs name their acceptor and the proposer count per column.

The P1b quorum counted replies, not distinct acceptors. A proposer whose check finds no leader re-sends the P1a of the *same* ballot, an acceptor answers every P1a it receives, and the election counted P1bs per `(ballot, column)` without naming the acceptor, so two replies from one acceptor made a quorum of two. This one is an LDFI counterexample: lose the first-round P1bs of the two acceptors holding `x`, let the third acceptor's first and second replies make the quorum, and `y` is proposed at the slot `x` was chosen in — a violation of agreement on correct Paxos in 48 hypotheses and on correct compartmentalized Paxos in 18. It was invisible until the harness author moved the body's end of finite failures after proposer 1's first check, so that the first-round P1bs became omission candidates; under the earlier cut the first round always arrived and the quorum was distinct acceptors' by luck of ordering. History §9.2 adds that loss is not even needed: two slow acceptors and one fast one, a timeout and a retry, and the fast one's two replies are a quorum. The fix keeps one P1b per `(ballot, acceptor)` before counting.

History §9.2's own assessment stands: all three are defects "of the kind the simulator finds given a scenario that drives a leader change; what hydrofrac added is the scenario's faults and, for the second and third, the report that named the schedule or the lost messages."

Two further findings made while writing harnesses are outside hydrofrac's contribution and are listed only so they are not re-attributed to it: `collect_quorum_with_response` was not a function of its input despite its `nondet!` justification, found by the simulator's exhaustive exploration with no faults (history §1.2), and Hydro's Maelstrom backend shared one untagged message type across every cluster-to-cluster channel, found by the first Maelstrom run of the Raft comparison (history §9.1).

Part III — What we learned about the search

1. Provenance granularity: which levels earned their keep

The question was whether finer lineage buys fewer hypotheses, and how much finer it needs to be. At the start of the last round the code had four levels: `Execution` (L0: every output of an execution depends on every input of it and of every earlier execution at its node), `Slice` (L1: L0 narrowed by a static backward slice from the producing sink, then the default), `Record` (L2: record provenance from the `with_provenance()` IR rewrite, each element carrying the delivery ids it was computed from, replaying the node at a `Prov::OPAQUE` record), and `Counterfactual` (L3: node-local replay with deliveries removed one at a time, latest first, keeping those without which the record disappears) (ablation header). The experiment ran the *same* search at all four levels on the same provenance-compiled program, over 2PC (`n ∈ {2,3,4}` × load `{1,4,8}` × crash `{0,1}` × CTP), ISR (two encodings × bug/correct × three explorations), reliable broadcast and delivery, Raft and core Paxos (commits c11ac0147a, f956b8c5b4, 6b2acd41d8, 31d3eec1e8).

L0 equals L1 in every cell of every protocol. On 2PC the columns agree across all 36 configurations; on ISR in every cell of both encodings; on broadcast and delivery in every row; on Raft to the digit (ablation §"Observations (2PC)", §"ISR", §"Reliable broadcast", §"Raft"). The slice never separated a record from the inputs of its execution. On a closure-encoded protocol it cannot — `raft_step` is one IR sink — and on the dataflow encodings both inputs of a member's tick feed the same outputs.

L2 pays exactly where there is background load or redundancy. On 2PC+CTP, L2's derivation stays at `4·n` dependencies and its certificate count is load-invariant at 13 / 18 / 23 hypotheses for `n = 2 / 3 / 4`, while L0's grows about nine hypotheses per background transaction, to 51 / 76 / 101 at load 8: 101 → 23 at the largest cell

(ablation §"2PC with cooperative termination"). The earlier "11 vs 73" reproduces as 18 vs 76 — 4.2× rather than 6.6× — deterministic over five repeats; the shift is uniform across levels and loads and is probably the harness, but this was not verified (ablation §"Observations (2PC)"). On ISR, clean, one crash: 31 → 12 (closure) and 31 → 18 (dataflow); on the bug 14 → 9 / 12. On echo broadcast: 15 → 10 at one crash and 73 → 32 at two, with 20–30% less wall time despite the bookkeeping, because execution lineage charges a member's delivery to *every* message in its tick batch (ablation §"Reliable broadcast"). On Paxos, L2 certified the correct variant in 195 and 230 hypotheses in two passes where L0 ran out at 256 (ablation §"Paxos", §"Final"). Reliable delivery is level-insensitive: one-leaf derivations.

L3 reproduced L2 wherever L2 applied, at two to ten times the cost. On 2PC, L3's count equals L2's on every certificate at 3–10× the wall time (11.2 s against 1.3 s at `n=4 load=8`); on broadcast, identical hypotheses, schedules and derivations at 2.7–3.5× (65,388 replays against 17,280); on Paxos, same-sized derivations at 1.6–2.3× (ablation §"Observations (2PC)", §"Reliable broadcast", §"Paxos"). Its one real win is the dataflow ISR bug in 3 hypotheses instead of 12, because its failure-free derivation is `{hb×3, observed}` and the leader's crash is hypothesis #2 (ablation §"ISR"). That does not pay for Raft, where replay is ~5 s a hypothesis (ablation §"Recommendation record").

The Raft exception. Raft is one `raft_step` closure per tick, so under the rewrite its outputs are opaque and L2 replays through the step — L2 *is* counterfactual lineage there, and L3 adds nothing. Derivations shrink 4.4× (46.4 → 10.5 leaves), but hypotheses go up: L0 certifies both variants in 59 hypotheses in 17–19 s; L2 runs out of a 64-hypothesis budget after 243–306 s and, given 160, certifies in 107 after 403 s. The over-approximate clause "everything delivered to the member" is hit by almost any fault set, so the universe saturates fast; the precise clauses each name a different path and must be defeated separately (ablation §"Raft"). A program that is closure-encoded end to end should pick `Execution` explicitly, and that is why `Execution` survives as a level (ablation §"Recommendation record").

The collapse. Commit `4ddea33f19` removed `Slice` and `Counterfactual`, with the re-measured two-level tables agreeing with the L0 and L2 columns in every cell. Commit `fafcd70fe3` then fixed the fallback the collapse had got wrong: a program compiled without `with_provenance()` had been silently downgraded to `Execution` (`Ldfi::effective_level`), when the right reading of a program with no tags is *every record opaque*, derived by replay. `Record` became `Precise` — record provenance where the program carries it, counterfactual replay where it does not, execution closure only where replay is inconclusive — and `effective_level` was deleted. On ISR compiled without provenance: the bug in 9 hypotheses instead of `Execution`'s 14, the certificate in 12 instead of 31, zero fallbacks; on 2PC+CTP `n=3 load=8`, 18 instead of 76, with 27 fallbacks because the coordinator's transactions arrive through an un-hooked `sim_input` and a solo replay never sends a `prepare` — the fallback doing its job; on `leader_merge_broadcast`, which the rewrite rejects, the same 2-hypothesis counterexample with 14 replays, 0 fallbacks and no pinned level (ablation §"Precise without provenance"). Replay is an oracle inside `Precise`, not a level: it runs at opaque records, at every record of programs without provenance, and at the observer for every hand-written view, which is why the `Execution` cells of the ISR tables show replays too.

2. The soundness bug the ablation caught

The first ISR grid showed L2 finding the unclean-election bug while L0, L1 and L3 **certified** it, on both encodings, under `fuzz 1` and `fuzz 8`; the repo's own `ldfi_finds_the_unclean_election_bug{,_dataflow}` failed under `LDFI_NO_PROVENANCE=1` (certificate in 14) and `LDFI_LEVEL=replay` (certificate in 6) (ablation §"Soundness bug", pre-fix table; commit f956b8c5b4). The diagnosis written down at the time — vacuous hypotheses adding no clause and being read as coverage — did not survive the solver's own trace. The

counterexample `X = {omit(repl[0→1]), crash(replica@0 after client-ack)}` was a superset of none of the vacuous sets the search saw. `X` was never proposed because **`X` was not a hitting set of the clauses the search had**: some clause claimed `POST` rested only on deliveries `X` does not kill, while the simulator shows `X` kills `POST`. Two unrelated gaps produced such clauses (commit 4c43956b4b).

Root cause 1 (L0, L1): dependency ordinals clamped at EFF. `Lineage::crash_ordinal` clamped a dependency's send ordinal to the sender's pre-`EFF` send count, assuming crash points lie before `EFF`; the simulator fired `crash(n after m)` wherever `m` was sent. In the omission world replica 0's acknowledgement was post-`EFF` (send #4 of `#0 repl[0→1] #1 repl[0→2] | EFF | #2 shrink #3 hb #4 client-ack #5 hb …`), every replica-0 dependency carried the clamped ordinal #2, and `killed_by` computed `position(client-ack) = 4 < 2`: the crash kills nothing of the clause, while in fact it kills the heartbeats #5–#7 that keep 0 the leader. L2 found the bug only by hypothesis order: its smaller failure-free clause made the crash hypothesis #2, before any omission-world clause existed. The fix made the ordinal exact and moved the pre-`EFF` clamp to where crash points are *proposed* (ablation §"Root cause 1").

Root cause 2 (L3): sequenced observations never traced. `Spec::observe_sequenced` enumerates before the observer's hooked batch, so the release payload is `bincode((usize, View))` while the wire bytes are `bincode(View)`; `payload_matches` never matched a view to ZK's send — the "32 untraced deliveries" on every ISR log line. Replay takes only matched releases as removable candidates, so at L3 the views were fixed facts with no lineage: derivation `{observed[0→]#2}`, two crash points, a certificate in 6. Accepting a serialized `usize` prefix made the derivation `{hb×3, observed[0→]#2}` and the bug appears at hypothesis #3 (ablation §"Root cause 2").

The alternative fix for root cause 1 — confining crash points to the window in the simulator — was tried as a one-line experiment and made the ISR bug unreachable at every level under that harness, because the acknowledgement was post-`EFF` in the omission world. The search was first made to agree with the simulator; then `4ddea33f19` adopted the simulator-side reading as Molly's model and moved the ISR harness's `EFF` past the first round so the acknowledgement lies inside the window in every world (ablation §"Root cause 1", §"Final"). Where history §10 says `EFF` "should not shape the design further", the current design disagrees: it is the fault model.

After the fix, verdicts agree at every level in every cell: ISR bug at L0 (14), L1 (14), L2 (12), L3 (3), pinned by `isr_replication::tests::ldfi_finds_the_unclean_election_bug_at_every_level`; every 2PC verdict and certificate count unchanged; every broadcast and delivery cell identical (ablation §"After the fix"). "The verdict is independent of the lineage level" is now a test, and it is the strongest argument for having run the ablation: a level grid is a soundness oracle for free, since the levels are supposed to differ only in cost. Vacuous-superset pruning was left as it was; nothing showed it changing a verdict (ablation §"Vacuous hypotheses").

3. Disjunction: why Molly's disjunctive provenance bought nothing here

Molly's lineage from one good run is a formula: a conjunction within a derivation, a disjunction across the alternative derivations of one fact, and the solver prunes every fault set that leaves some alternative intact without running it. Hydrofrac kept one witness per record — at a dedup, the first copy's — and rediscovered alternatives by simulation. Commit `73a5db9c05` recovered the disjunction behind `Ldfi::disjunctive_provenance(bool)`: `Prov` became a shared, lazily resolved handle over `Leaf(delivery) | All(children) | Alts(children)`; pass-through operators share the handle, combinations build `All`, and a `map(rebox)` before every in-tick `unique` gives each arrival a fresh `Alts` cell that `Tagged::eq` fills with the duplicate's alternatives (lazy and shared because DFIR's `unique` emits the first copy before its duplicate arrives). The formula resolves to a minimal DNF under `max_witnesses` (default 8; an overflowing conjunction falls back to the union witness, an overflowing disjunction

keeps its first `cap` alternatives, both sound). With a cap of 1 the resolution is exactly the old single derivation, and `two_pc_ldfi::tests::single_derivation_search_is_unchanged_by_the_provenance_formula` pins the thirteen hypotheses of the exhaustive 2PC+CTP `n=2 load=4` search, in order, to the pre-change sequence (ablation §"What was built").

I expected this to reduce hypothesis counts. It did not, on any protocol, in any cell, in three measurement passes. On 2PC and 2PC+CTP every verdict and certificate count is identical, because no run exhibits two derivations of one fact: a participant queries its peers only after a timeout while uncertain, and the peer path fires only in worlds where the coordinator crashed, where the participant hears *only* the peer. Alternatives exist across worlds, not within a run, and Molly's single-run lineage would see what hydrofrac's does. On ISR, in both encodings, clauses per run are identical to two decimals across five repeats, which is only possible if no record resolved to more than one witness. On Paxos the P2b quorum is a `fold`, a conjunction (ablation §"Where the alternatives are", §"Complete grid: ISR, broadcast, delivery", §"Core Paxos").

Echo broadcast is the one protocol with in-run alternatives, and there the hypothesis count went **up**: 10 → 11 at one crash and 32 → 39 at two under exhaustive exploration at `n=3`, reproduced exactly in two passes; 15 → 21 (+40%) at `n=4` under `fuzz 8`, identical over five repeats, with the witness cap of 8 overflowing for the first time in the corpus (156–176 times per search; four members give up to four witnesses per delivery and $4^4 > 8$). The diagnostic, reconstructed from the searches' own records with every hypothesis re-checked as a minimal hitting set, explains the arithmetic. The disjunctive failure-free run yields 16 derivations instead of one, and its clauses prune the two `crash(broadcaster after initial[→k])` hypotheses the single search ran — Molly's pruning, and it is real. But every alternative's deliveries become dependencies, and their senders' early sends enter the universe: `crash(member@1 before any send)`, `crash(member@2 after echo[2→0])`, `crash(member@2 before any send)`, named by echoes from a higher-numbered member to a lower-numbered one, which never appear in any single-witness derivation because the lower-numbered member's first copy is always the broadcaster's `initial`. Saved 2, added 3; at two crashes saved 12, added 19. A blind search over all twelve fireable crash points confirmed the goal holds under every one (ablation §"Diagnostic"; commits aa10020989, 0bbf22d080).

The deeper reason is structural. Hydrofrac's fault universe is grown lazily from the derivations it has seen; a crash point is proposed only once some derivation depends on the send that names it, so a fault that kills no observed dependency is never tried. That is itself most of the pruning Molly gets from the formula. Molly's universe is every message of the run, fixed up front, so the disjunction is what keeps *its* search from running every one of them; hydrofrac gets that from the universe instead. The DNF certificate is relative to a larger universe, so it is a stronger certificate at more runs, not a cheaper one (ablation §"Observations" under Disjunctive provenance). The flag stays off. The follow-up worth one experiment is DNF cover constraints over the single-derivation universe: keep every alternative as a clause but propose crash points only from the first-witness derivations, dropping (soundly) any clause the universe cannot hit, which would keep the two pruned broadcaster hypotheses without admitting the three new member ones (ablation §"Status").

4. Aggregate-local counterfactual witnesses

The disjunction only ever fired at a dedup, and the corpus's redundancy is quorum-shaped: a `fold` (a count) over `n` inputs followed by a threshold predicate, whose output is tagged with the conjunction of all `n`. "Any `k` of `n`" is lost at the aggregate, in hydrofrac as in Molly. Commit `2f156b6335` recovers it behind `Ldfi::aggregate_witnesses(bool)` (`LDFI_AGGREGATE=1`, default off). Under the mode a `fold` / `fold_keyed` accumulator's cell is a `Node::Agg`: the conjunction of every input (what it resolves to, so every unaware reader sees the old formula) plus each input's value and provenance, up to `AGGREGATE_MAX_INPUTS = 10`, and a type-erased

re-fold built from fresh instances of the fold's own `init` and `acc`. A downstream `filter` or `filter_map` re-runs the fold on all `2^n` subsets in both input orders and applies its own predicate; the output's provenance becomes the disjunction of the **minimal passing subsets** if and only if, on this run's inputs, the predicate is upward-closed, order-independent and passes the full set — checked empirically, since the IR carries no commutativity annotation. Otherwise the conjunction stays and the skip is counted by reason. The empty subset is never a witness; keyed↔flat casts re-key and project the re-folded value; every other value-changing operator strips the cell to its conjunction so no predicate re-evaluates an aggregate through a value it did not see (ablation §"What was built" under Aggregate-local witnesses). (The caps 10 and 8 are discussed with the hypothesis budget in §5.)

Where it fires: Paxos's P2b commit quorum (`collect_quorum(p2b, 2f+1)`: `fold_keyed → entries → filter_map(success >= min)`), with zero skips — but a second witness exists only in schedules that batch all three acknowledgements into the tick in which the count first reaches `min`, since `collect_quorum` emits a slot once. It does **not** fire on the P1b leader-election quorum: `collect_quorum_with_response` selects the *responses* of qualifying keys by `anti_join` against the keys below threshold, provenance does not flow through a negation, and the count's predicate never sits on the records' path; the same holds for any quorum written as "the responses of keys whose count passed". On 2PC (`count >= num_participants`, all-of-n) every output is `unchanged`; echo broadcast has no fold on the path (ablation §"Where it fires in the corpus").

The numbers do not separate it from noise. Paxos bug at cap 256, three repeats: `precise` 23–67, `disjunctive` 21–43, `aggregate` 21–53 hypotheses to the counterexample, draw-dominated. Paxos correct exhausts 256 in every column and every repeat. At cap 400, one pass each, `aggregate` certified at 339 against `precise`'s 393 while `disjunctive` ran out at 400 — within the variance `precise` has shown across this document (203, 230, 256+, 393) — and mean derivation size is unchanged at 19.6–20.4, because the goal's derivation is dominated by leader-election and payload deliveries. 2PC and echo broadcast are identical to their `precise` cells hypothesis for hypothesis; the `hydro_std` quorum unit shrinks its derivations from six deliveries to four at the same 28 hypotheses. Wall time is within noise (ablation §"Paxos", §"Observations" under Aggregate-local witnesses).

It stays off. What would make it worth turning on: a protocol whose quorum predicate sits on the records' path (a count, not an anti-join of responses), whose alternatives appear in the failure-free run rather than only in favourable batchings, and a search long enough for a smaller clause to matter — then a multi-pass measurement with several sampled schedules, since one pass at ±50% draw variance cannot show it.

5. The builder's experimental knobs, and the three caps

Everything on `Ldfi` that Part I did not mention is an experiment or a diagnostic, and this is where they live:

Knob	Default	Env	Effect
`.level(Level::Execution | Precise)`	`Precise`	`LDFI_LEVEL=exec|precise` (`record` is an alias)	which lineage oracle derives a record (commit fafcd70fe3)
`.disjunctive_provenance(bool)`	off	`LDFI_DISJUNCTIVE=1` via `.disjunctive_from_env()`	follow every witness of a deduplicated record (commit 73a5db9c05)
`.aggregate_witnesses(bool)`	off	`LDFI_AGGREGATE=1` via `.aggregate_from_env()`	minimal passing input subsets at threshold folds (commit 2f156b6335)

Knob	Default	Env	Effect
`.max_witnesses(k)`	8	—	cap on witnesses per record; overflow falls back to the union witness, soundly
`.with_traces()`, `.with_provenance_graphs()`	off	—	retain every schedule's `Trace`; record a `ProvGraph` beside each derivation (viz)
`.exploration(Exploration::Fuzz { instances })`	`Exhaustive`	—	sampled schedules; a certificate is then not a proof
`.strategy(Strategy::Blind)`	`Lineage`	—	the ablation: same universe, no clauses
`.max_hypotheses(n)`	256	—	`Outcome::Budget` when exceeded
gallery tests	no-op	`MOLLY_VIZ_OUT=<dir>`	`gallery_*` tests write `.dot` and `report.md` (gallery README)
ablation harnesses	no-op	`LDFI_ABLATION_OUT=<path>`	`level_ablation_*` tests write their Markdown grid (ablation header)

`LDFI_PROV_DEBUG`, `LDFI_SOLVER_DEBUG`, `LDFI_LINEAGE_DEBUG` and `LDFI_REPLAY_DEBUG` print, respectively, records with more than one witness, every clause and hypothesis, the lineage log and the replays (ldfi.rs).

Three of these are numeric caps, and each exists for a different reason. The hypothesis budget exists because the Molly loop has only two natural exits — the ILP goes infeasible (a certificate) or a hypothesis violates — and because hydrofrac grows the fault universe lazily, a search that keeps adding new crash points can keep going. The budget is the third exit, reported as `Outcome::Budget` rather than dressed up as a certificate. It is a time bound, not a termination guarantee: the universe is finite (a bounded body sends finitely many messages, the crash budget and one crash per node bound the crashes, and a no-good constraint per tried set means no hypothesis repeats), so the ILP would go infeasible eventually. What actually hits the cap is slow or noisy, not infinite. Raft at `Precise` runs out at its budget of 64 after 243–306 s, about 4 s a hypothesis, while `Execution` certifies the same property in 59 (ablation §"Raft"); correct Paxos at one sampled schedule per hypothesis certifies at 203–230 on one pass and hits 256 on another, which the ablation calls "a budget effect of the draw, not a verdict disagreement"; and the aggregate-witness experiment needed a cap of 400 to finish at all (certificate at 339 against 393) (ablation §"Core Paxos", §"Aggregate-local counterfactual witnesses"). The witness cap exists only because of disjunctive provenance: when a record has several witnesses, derivations multiply as a cross product through conjunctions, and the ablation measured $4^4 = 256$ candidates for one goal on four-member echo broadcast. Over the cap, a conjunction falls back to the single union witness (a superset of a real witness: sound, less pruning) and a disjunction keeps its first `cap` alternatives (dropping alternatives is sound); both are counted as `witness_overflows`. The cap was never reached at `n = 3` on any protocol and was hit 156–176 times per search at `n = 4` echo (ablation §"Disjunctive provenance", §"Complete grid"). With disjunction off, as shipped, a cap of 1 reproduces the old single derivation bit for bit, so the number does not matter. `AGGREGATE_MAX_INPUTS` bounds the 2^n re-folds (run twice) that test a fold's predicate for upward-closure; past ten inputs the cell overflows and the record keeps its conjunction (provenance.rs). None of the three values is justified by a measurement in any source; they read as round numbers. [NEEDS AUTHOR REASONING: were 256 / 8 / 10 chosen by any measurement, or are they placeholders to be tuned once the budget verdict is seen in anger?]

6. Fault point and fault model

The fault point is the sender because the first version put it at the consumer and paid for it. Round 1 used the simulator's scripted `BatchHook` — a standing `OmissionPolicy` on the receiving batch, channels registered by the harness — because interposition there was free. The cost: a `tee`d channel omitted at one consumer was delivered at the others; hook-free paths could not be faulted; crashes needed a separate mechanism in the send half; every faultable batch was a hook parameter in the protocol's signature; and every certificate rested on the author having registered every channel. All followed from one choice: a receiver-side test seam is not a model of the network (history §11). The filter on every `Network` send, with the transport type as the fault model, made every channel a fault point by construction.

7. The specification is a Hydro flow

`Spec` builds an observer at a dedicated process and compiles it with the program. Observations arrive over an ordinary `fail_stop` channel and are presented in the observer's tick as the set of facts so far; the specification's views are bounded `Optional`s in that tick, and a view holds if it is non-empty at the end of the run, as a Datalog goal holds at fixpoint. Because the observer is a Hydro flow its outputs have lineage like any other output, and that is what the search negates (history §6.5). The observer's ticks run eagerly, outside the schedule choice: it only consumes, its interleaving is unobservable to the program, and exploring it multiplied schedules 64× before the rule (history §6.5). For `Spec::all_labels` the observer's input is a slot filled at compile time by an IR pass (`sim::observe`) that lists the labelled collections, turns each labelled node into a `Tee` as a `.clone()` would, maps the second branch to `(collection index, bincode of the element)` and sends it to the observer; the pass runs before network compilation and before the provenance rewrite, so the taps are traced like any operator (hydrofrac.md §"The specification is itself a Hydro flow"). `FACTS` comes from a failure-free probe before the first hypothesis; a probe that yields nothing is `Outcome::Vacuous` — a verdict about the harness, as the 2PC cooperative-termination rows under `all_labels` show when no peer is ever queried in the failure-free run (hydrofrac.md §"Every label of ISR replication, two-phase commit, and ABD").

Part IV — Doors

Closed doors. Sources: `history-hydrofrac-development.md` (history §n), `history-hydrofrac-negation.md` (negation §n), `ldfi-on-sim-hooks.md` (hooks §"title"), `ldfi-level-ablation.md` (ablation §"title"), and commits.

Consumer-side simulator hooks as the fault point

- **Tried:** round 1; standing `OmissionPolicy` / `PassthroughPolicy` on the receiving `BatchHook`, channels registered by the harness (hooks §"Prototype status").
- **Closed because:** a test seam, not a network model: `tee`d outputs, hook-free paths, crashes as a separate mechanism, hook parameters in protocol signatures, certificates resting on complete registration (history §11).
- **What it bought us:** the sender-side filter on every `Network` send with the transport type as the fault model; hooks survive as optional lineage precision and as what replay feeds (history §5.5, §11.5).
- **Reopen only if:** never; the sender is where Molly defines omission and crash.

Crash points named by send count; message identity by occurrence counting

- **Tried:** `crash(node after k sends)` (hooks §"Two soundness problems", 1); `(channel, from, to, payload, k)` with retransmissions as distinct faults (hooks §"Message identity and time").
- **Closed because:** under another hypothesis the send sequence shifts, `send #7` names a different message, and 2PC at load 2 was certified with the bug present.
- **What it bought us:** `crash(node after m)` with `m` identified by `(channel, sender, destination, bytes)`; a node whose `m` is never sent survives; a byte-identical retransmission is the same omission (hydrofrac.md §"Transports supply the fault model"). [NEEDS AUTHOR REASONING: was "an identical retransmission is the same fault" accepted deliberately? A protocol that retries with identical bytes can be tested for "all copies lost" but never for "first lost, retry delivered".]
- **Reopen only if:** a target retransmits byte-identical payloads and the bug needs one copy lost and another delivered.

Every observed send as a crash candidate

- **Tried:** after message-named crashes, L2 certified ISR at load 2 because the leader's crash *between* two acknowledgements was in no goal derivation; the first fix admitted every observed send (hooks §"Two soundness problems", 2).
- **Closed because:** singleton hypotheses became the number of sends, independent of lineage (2PC load 8: 46 instead of 11).
- **What it bought us:** Molly's answer — the crash right after each send in a `PRE` derivation is a *preserve point*; universe = clause candidates `∪` preserve points.
- **Reopen only if:** never.

Exploring the observer's schedule

- **Tried:** the observer as an ordinary location whose ticks the explorer interleaved.
- **Closed because:** 64× more schedules on broadcast for an interleaving the program cannot observe (history §6.5).
- **What it bought us:** eager observer ticks outside the schedule choice.
- **Reopen only if:** never.

The static slice (`Level::Slice`, L1)

- **Tried:** round 1's `sim/slice.rs`, a per-sink backward slice narrowing the execution closure; the default until the collapse (ablation header).
- **Closed because:** L0 = L1 in every cell of every protocol measured (ablation §"Recommendation record"). Removed in commit 4ddea33f19.
- **What it bought us:** the diagnosis that record provenance, not slicing, keeps counts flat under load; `sim/slice.rs` survives to name channels and hook inputs.
- **Reopen only if:** never as a level.

Counterfactual replay as a level (`Level::Counterfactual`, L3)

- **Tried:** node-local replay as the top rung, falling back to L2 (ablation header).
- **Closed because:** identical derivations to L2 wherever L2 applied, at 2–10× the cost; one win (dataflow ISR bug 3 vs 12) that does not pay for Raft at ~5 s a hypothesis (ablation §"Recommendation record"). Removed in commit 4ddea33f19.
- **What it bought us:** replay as the oracle *inside* `Precise` for opaque records and programs without provenance (commit fafcd70fe3).
- **Reopen only if:** never as a level.

Downgrading no-provenance programs to `Execution` (`Ldfi::effective_level`)

- **Tried:** commit 4ddea33f19, as part of the collapse.
- **Closed because:** a program with no tags should be *every record opaque*, derived by replay; the downgrade gave `leader_merge` the coarsest lineage and ISR 31 hypotheses instead of 12 (ablation §"Precise without provenance"). Removed in commit fafcd70fe3.
- **What it bought us:** `Precise` as the uniform default; harnesses need not know whether their program carries provenance.
- **Reopen only if:** never.

Disjunctive provenance on by default

- **Tried:** commit 73a5db9c05, three measurement passes (ablation §"Disjunctive provenance", §"Complete grid" ×2).
- **Closed because:** no reduction on any protocol; +10% / +22% / +40% hypotheses on echo broadcast, because alternative witnesses add new crash points (ablation §"Diagnostic").
- **What it bought us:** the formula representation (what `Precise` reads its single derivation from); the insight that a lazily grown universe already does Molly's pruning; the DNF-over-single-universe idea.
- **Reopen only if:** a protocol exhibits in-run alternatives whose deliveries are sends the single path already names.

Aggregate witnesses on by default

- **Tried:** commit 2f156b6335; Paxos × 3 repeats at cap 256, one pass at cap 400.
- **Closed because:** draw-dominated; 339 vs 393 in one pass is within Paxos's ±50% variance; the P1b quorum is unreachable through its `anti_join` (ablation §"Observations" under Aggregate-local witnesses).
- **What it bought us:** which quorum shapes carry recoverable "any `k` of `n`" and which do not.
- **Reopen only if:** see Part III §4.

The syntactic polarity check

- **Tried:** round 4, `polarity.rs`: a walk over the observer's dataflow grading each view `Monotone` / `NonMonotone` / `Unknown` (negation §5; history §12).

- **Closed because:** it could only approximate (the positive agreements graded `Unknown`; `first()` is an unclassifiable `scan`), and once the specification was generated and sound by form there was nothing to classify. Removed in round 5.
- **What it bought us:** the grades as words — within the fault model for the generated specification, RELATIVE for every hand-written one.
- **Reopen only if:** hand-written specifications become a product rather than an escape hatch.

Counterfactual-only lineage at the observer

- **Tried:** `spec_view` taking the replay-derived set alone.
- **Closed because:** the false ISR certificate in 6 hypotheses on the final-view `POST` (negation §4).
- **What it bought us:** the union of counterfactual and recorded derivations; the statement that the inference holds only for monotone views.
- **Reopen only if:** the view is monotone by construction, which the generated specification is.

Clamped crash ordinals; untraced sequenced observations

- **Tried:** found, not tried: the two clause-soundness gaps that certified ISR's bug at L0/L1 and L3 (ablation §"Soundness bug").
- **Closed because:** commit 4c43956b4b made the ordinal exact, moved the clamp to proposal, and taught `payload_matches` the `usize` prefix.
- **What it bought us:** `ldfi_finds_the_unclean_election_bug_at_every_level`; the level grid as a soundness oracle.
- **Reopen only if:** never.

The staged Raft body ("17 hypotheses")

- **Tried:** rounds 1–4 reported 25, 20, then 17 hypotheses with a body that arranged everything the bug needed except the two lost messages (history §9).
- **Closed because:** the body was part of the claim and its author knew the bug. Withdrawn.
- **What it bought us:** a generic body shaped only by what correct Raft needs; the bug in 90, correct Raft in 235.
- **Reopen only if:** never.

Tuning the harness body instead of `PRE`

- **Tried:** round 5: four Raft bodies "violated" durability on the correct protocol (no post-`EFF` election timer; no request of the new term; a timer spent on a re-armed suppression; too few heartbeats for the linear back-off), each lengthened; the fourth also changed the protocol (history §9).
- **Closed because:** `PRE` = "some member committed" is true at the first commit, so a run too short to catch up read as a bug: a specification error treated as a scheduling problem (history §1.1).
- **What it bought us:** `PRE` as a convergence witness. The Raft harness is not yet on this pattern (history §12, item 0).
- **Reopen only if:** never; fix `PRE`, not the horizon.

Silent certification when `PRE` never held

- **Tried:** rounds 1–4: a failure-free run with `PRE` false contributed `{}`, the ILP was infeasible, outcome `Certificate` (history §9).
- **Closed because:** the first generic Raft body sent requests to non-leaders and certified in one run.
- **What it bought us:** `Outcome::Vacuous`.
- **Reopen only if:** never.

Letting a counterfactual replay panic

- **Tried:** found: Raft's leader indexed past its log under replayed inputs and the panic crossed the dylib boundary and aborted the process (history §9).
- **Closed because:** a replay that cannot run is no evidence, and the abort took the search with it.
- **What it bought us:** `sim::runtime::erase_guarded`; a replay panic is inconclusive and keeps the delivery. Hit 624 times in 365,000 Raft replays.
- **Reopen only if:** never.

Agreement as the Raft property

- **Tried:** rounds 1–3: "no index holds two committed entries" (hooks §"A Raft bug").
- **Closed because:** agreement is the absence of a conflict; the staged bug was found only because the same omission also displaced a recorded commit (history §6.5, §10).
- **What it bought us:** durability as the Raft property; agreement belongs to the simulator's own tests and Maelstrom.
- **Reopen only if:** never for LDFI.

A lattice-aware generated specification for `Singleton` outputs

- **Tried:** proposed for ABD's register: observe the final value per correct member, `POST` equality (history §12, item 0).
- **Closed because:** checking the value re-proves the type system's conclusion; the tap goes on the input stream (hydrofrac.md §"Next steps", 1; commit 7d58ad20e0).
- **What it bought us:** a clean class boundary: dissemination, never computation.
- **Reopen only if:** a scalar is labelled over an unlabelled input whose convergence nothing vouches for.

The undischarged-premise lint

- **Tried:** round 5: every search printed every `EventualConsistency` source and whether the specification observed it (history §1.2).
- **Closed because:** a report cannot fail a test; the direct route was to search every label (auto-labels spec §1).
- **What it bought us:** `Spec::all_labels` / `search_all_labels`; `LabelResults` is the lint.
- **Reopen only if:** never.

Maelstrom and partition nemeses for the election-restriction bug

- **Tried:** a Raft-backed `lin-kv` workload; twelve runs, 29 minutes, partitions and latency nemeses, ~1,900 elections (history §9.1).

- **Closed because:** the bug needs *selective* loss, and a partition drops everything on a link, so the isolated node falls behind and the index-only check correctly denies it. A premise is discharged by a completeness claim, not by "ran 29 minutes, no anomaly".
- **What it bought us:** the sharp statement of what hydrofrac is for; a real bug in the Maelstrom backend.
- **Reopen only if:** a nemesis drops individual message identities.

Measurement mistakes

- **Tried:** round 1 charged LDFI 4–8 schedules per hypothesis against one instance per random fault set, and the random baseline visited nodes in registration order, spending its crash budget on the coordinator half the time (hooks §"Ablation").
- **Closed because:** hooks under a policy draw no entropy, so one schedule per hypothesis gave every verdict of eight on protocols without schedule-sensitive bugs; the baseline was lucky, not uninformed.
- **What it bought us:** the corrected grid (lineage beats blind everywhere, 2–7× on certificates); the retraction of "random wins instances-to-bug"; Raft keeps four schedules because its outcome depends on the interleaving (history §5.5). The last round added two rules: wall seconds at load 20–30 are not benchmarks, and ≤ 4 concurrent exhaustive searches is the ceiling on a 15 GiB host (ablation §"Complete grid: ISR, broadcast, delivery").
- **Reopen only if:** never; but re-measure before quoting a Raft speed-up.

Why most doors closed

Nearly every entry above fell against one of three walls, and the last round added a fourth.

The wrong fault point. A fault injected anywhere but the sender, or named by anything but the message, is not the fault Molly reasons about: its identity shifts between hypotheses, or it misses channels, or it explores a dimension the program cannot see. Consumer-side hooks; crash points by send count and occurrence-counted identity; every observed send as a crash candidate; exploring the observer's schedule; Maelstrom's partition nemeses.

Unsound inference on a non-monotone view, or on an unsoundly attributed derivation. Every false certificate came from here. Counterfactual-only lineage at the observer; the polarity check; agreement as the Raft property; the clamped ordinals and untraced observations; letting a replay panic.

The harness doing the protocol's job. When the body arranges what the protocol should earn, the verdict is about the author. The staged Raft body; tuning the body instead of `PRE`; silent certification when `PRE` never held.

Finer provenance does not pay once the search universe is lazy. The slice, counterfactual as a level, disjunctive provenance and aggregate witnesses all hit this one. Each gives a more exact account of why a run succeeded; none changes which faults the search has reason to try, because hydrofrac proposes a fault only when some derivation depends on the send that names it, and the coarse account already names those sends. Where the finer account named *more* sends (the echo alternatives), the universe grew and the search ran longer for a stronger certificate. The one place finer lineage paid — record provenance over execution lineage under background load — is where the coarse account charged a record to deliveries it did not depend on, and that is already the default. The `effective_level` downgrade is this wall's mirror: coarser lineage where replay was free.

Three entries fit none: the lattice-aware specification and the undischarged-premise lint were about the scope of the claim, and the measurement mistakes about how it was measured.

Doors never opened

- **Delay and reordering.** Expressible at the filter as hold-instead-of-drop; reordering is the schedule explorer's dimension (history §5.6).
- **Partitions as a primitive.** Omission sets already express them, and partitions miss selective-loss bugs (history §9.1).
- **Byzantine behaviour, duplication.** Outside Molly's vocabulary; duplication has no simulator hook (hydrofrac.md §"Limitations").
- **`cut_after(n)` on a `fail_stop` channel.** Nothing has needed a reliable channel to fail other than by its sender's crash (history §5.6).
- **The filter under `deploy`.** One OS-chosen interleaving per run, no determinism, replay or barriers: Molly's fixed schedule or worse (history §11.3).
- **A receive-side source for hook-free replay.** Would let `BatchHook` drop out of lineage entirely; deferred, since replay reproduces every hooked node and falls back soundly elsewhere (history §11.3).
- **Why-not provenance by Molly's De Morgan rewrite.** The simulator answers the counterfactual question without knowing how the view was computed (negation §2–3).
- **Exhaustive schedules for Raft; blind and random on the generic Raft body; a coverage-guided fuzzer baseline; Raft under load.** Each an hour to a day of compute not yet spent (history §9, §10).
- **Raft `raft-bug` at `Precise` with a 256 cap in the shrunken ablation configuration**, to settle whether the L0 certificate on the bug variant there is a sampled-schedule miss (ablation §"Raft").
- **The disjunction grid on 2PC `n ≥ 3` CTP, `n=4`, Paxos-correct and Raft**, lost to the out-of-memory reboot (ablation §"Complete grid: 2PC, Paxos, leader_merge").

Open doors

- **Scalar labels checked at their input stream.** A labelled `Singleton` / `Optional` / `KeyedSingleton` is a function of a stream the types already vouch for; the tap goes on the labelling operator's input and the verdict is reported under the scalar's site. ABD's register (`abd.rs:165`) is the first case (hydrofrac.md §"Next steps", 1).
- **Convergence witnesses in `PRE`.** So an unconverged run is vacuous rather than a counterexample; the Raft harness is the first candidate and needs the latest-heartbeat acknowledgement as an output (hydrofrac.md §"Next steps", 2; history §12, 0).
- **Decision-trace replay.** Record every nondeterministic decision and replay a prefix with one changed: the general oracle of which node-local replay is the coarse instance. It does not by itself repair the inference for non-monotone views (negation §3; history §12, 1).
- **Raft's `committed` under the generated specification.** The in-class form of the durability test, with the sound grade; about an hour per search, never run (hydrofrac.md §"Next steps", 4).
- **Membership change.** Nothing on this branch changes configuration while running; the shape is written down — agreement across an epoch boundary, a crash during reconfiguration, membership events as a driven input (history §12, 2).
- **Legible Lamport diagrams for wide protocols.** `sim::viz::lamport` spaces lanes by the width of the longest message label, so a seven-node multi-Paxos counterexample with payloads like `(Ts { round: 1, writer: 0 }, 0, 1, Some(8))` is ~36 inches wide at a 9pt label and unreadable on a page at any scaling (the print renders in `molly-viz-samples/ec/PRINT.md` reach 3.9pt). The renderer should abbreviate payloads — an index into a legend, or a hash — so lane spacing is set by node count, not by the longest `Debug` string; until then the inventory PDF gives those figures oversize pages.

- **Crash budget `n - 2` by default for `all_labels`.** The generated property needs at least two correct members to be non-trivial; the default should be `n - 2` of the checked cluster, which needs a per-cluster `crash_budget` in `Ldfi`. Re-run the multi-Paxos labels at that budget (Part I §3).
- **DNF cover constraints over the single-derivation universe.** The one disjunction experiment left (ablation §"Status").
- **Replay cost on long runs.** Caching replays across hypotheses whose scripts share a prefix; a Raft hypothesis late in the certificate search is three to four minutes, almost all replay (history §12, 4).
- **The unmerged ablation knobs.** `LDFI_ABLATION_CTP` / `LDFI_ABLATION_LEVELS` for the 2PC harness (`017e8877af`, bookmark `sandbox-a76a3420-…`) and a parametrised `leader_merge_search` (`0157230103`, bookmark `sandbox-86fabe57-…`) are each one test-only commit on top of `2f156b6335`, written for the disjunction grid the reboot cut short.
- **The old goal API.** `goal`, `goal_all_members`, `goal_witnessed`, `Run::witness` / `require` are compiled and unused by any harness (history §10). [NEEDS AUTHOR REASONING: kept as an escape hatch on purpose, or to be deleted so no new harness is written against it?]

Status and housekeeping

`2f156b6335` is 34 commits past `9d0ec39c92`, a squash whose tree equals `ldfi-hydrofrac` (`7d58ad20e0`, 2026-10-18). Those 34 commits are all of `sim::viz`, the four ablation harnesses, the soundness fix, the Molly fault model, the level collapse and the Precise rename, disjunctive and aggregate provenance, and the ablation document. Anything written against `ldfi-hydrofrac` describes four levels, a `Slice` default, post-`EFF` crash points that fire, and no visualizations. The merge base with `main` (`90d9d26736`) is `99dcd3de68` (2026-08-11); 80 commits on the hydrofrac side, 75 on `main`.

`sandbox-e20764ef-…` (`b56e36ee15`) is a byte-identical tree to `2f156b6335` and can be dropped. `sandbox-86fabe57-…` (`0157230103`) and `sandbox-a76a3420-…` (`017e8877af`) are each one test-only commit on top of `2f156b6335` (above).

The hydrofrac line's first commit (`cd6cae7fc6`, "sandbox init") removed the `sedge`, `sedge_macro`, `cress` and `cress_config_macro` crates that its base (`4de7c45e41`) carried; `main` has none of them either. Whatever branch carries sedge/cress will conflict with the hydrofrac line at the workspace `Cargo.toml` and `Cargo.lock`, and that has to be resolved before either merges toward `main`.

`history-hydrofrac-development.md` carries a one-line header warning that its specifics may not match the code. It should say more loudly that it describes the state before the last round — four levels, `Slice` as default, crash points firing anywhere, `EFF` as "a leftover that should not shape the design" — or be marked archival and left alone. `molly-viz.md` §"Limits" still says alternative derivations are not drawn, which is true of the renderer but no longer of the data under `LDFI_DISJUNCTIVE=1`. The gallery files carry the suffix `record` where a regenerated gallery writes `precise` (gallery README).